

# Chapter 6 – Multi-Agent Systems

---

## Part 1 – Multi-Agent Fundamentals & Architectures

---

---

### 1. From Single Agent to Multi-Agent

---

*So far, we have mainly built:*

```
graph TD; User --> Agent1[Agent]; Agent1 --> Tools; Tools --> Agent2[Agent]; Agent2 --> Answer
```

User  
↓  
Agent  
↓  
Tools  
↓  
Agent  
↓  
Answer

*A single agent may have access to:*

RAG

Weather API

Database

ML Model

Market API

*This is enough for many applications.*

*But imagine a more complex task:*

Research the best crop,  
analyze weather,  
check market conditions,  
compare profitability,  
and create a final recommendation.

*One agent can technically do everything.*

*But as responsibilities increase, we may separate them into specialized agents.*

Crop Agent

Weather Agent

Market Agent

Research Agent

*This leads to:*

# Multi-Agent Systems

---

---

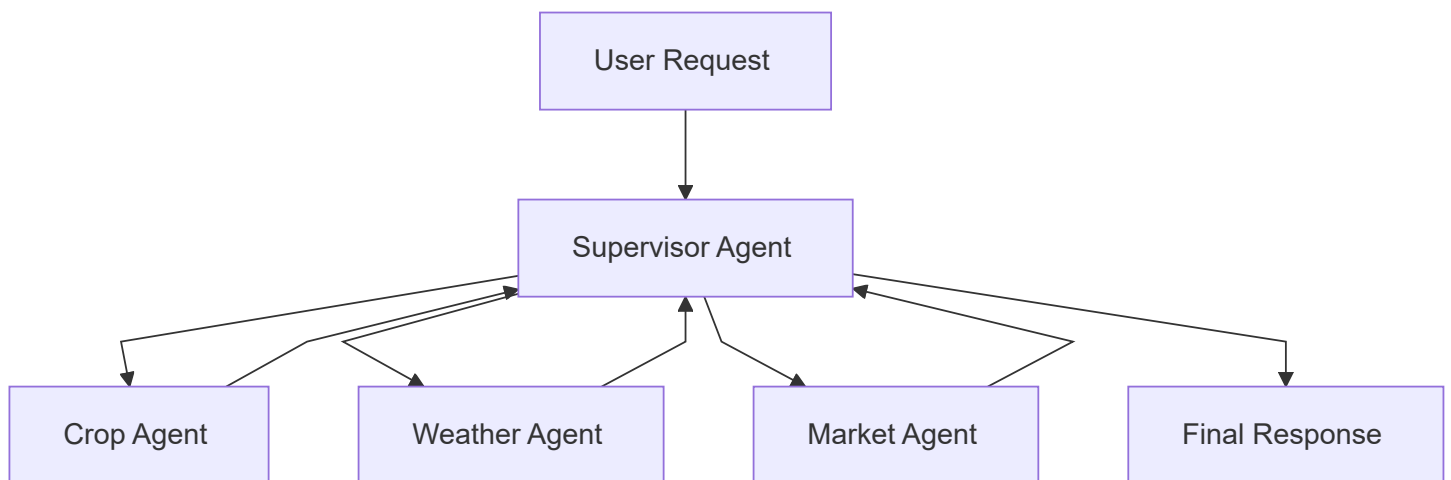
## 2. What is a Multi-Agent System?

---



A **multi-agent system** contains multiple specialized agents that collaborate or coordinate to complete a task.

Example:



Each agent has a specific responsibility.

---

## 3. Why Multiple Agents?

---

Suppose one agent has:

25 tools

Multiple databases

Several prompts

Many responsibilities

*Its decision space becomes large.*

*Instead:*

Weather Agent

→ Weather tools

Market Agent

→ Market tools

Crop Agent

→ Crop tools

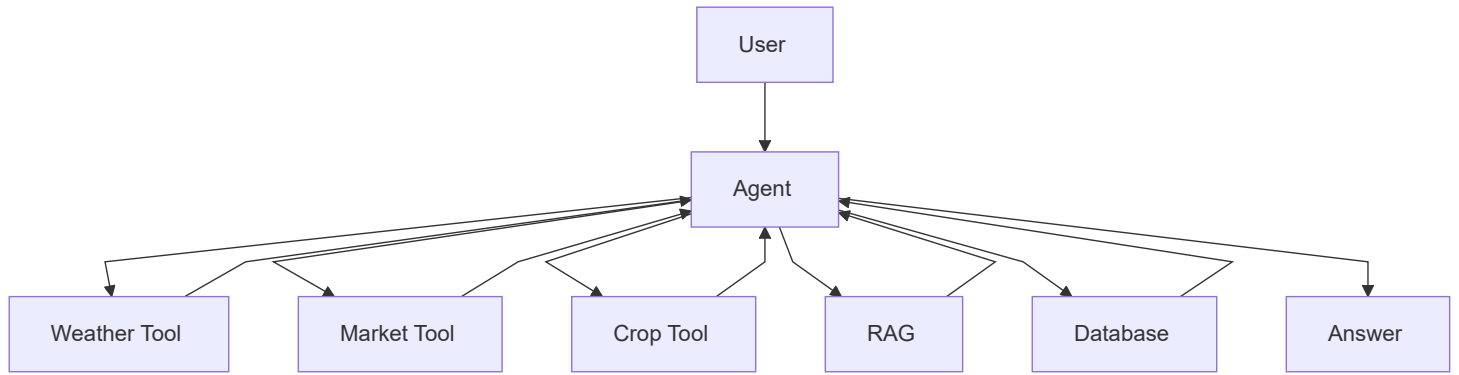
*Each agent gets a smaller, focused set of capabilities.*

---

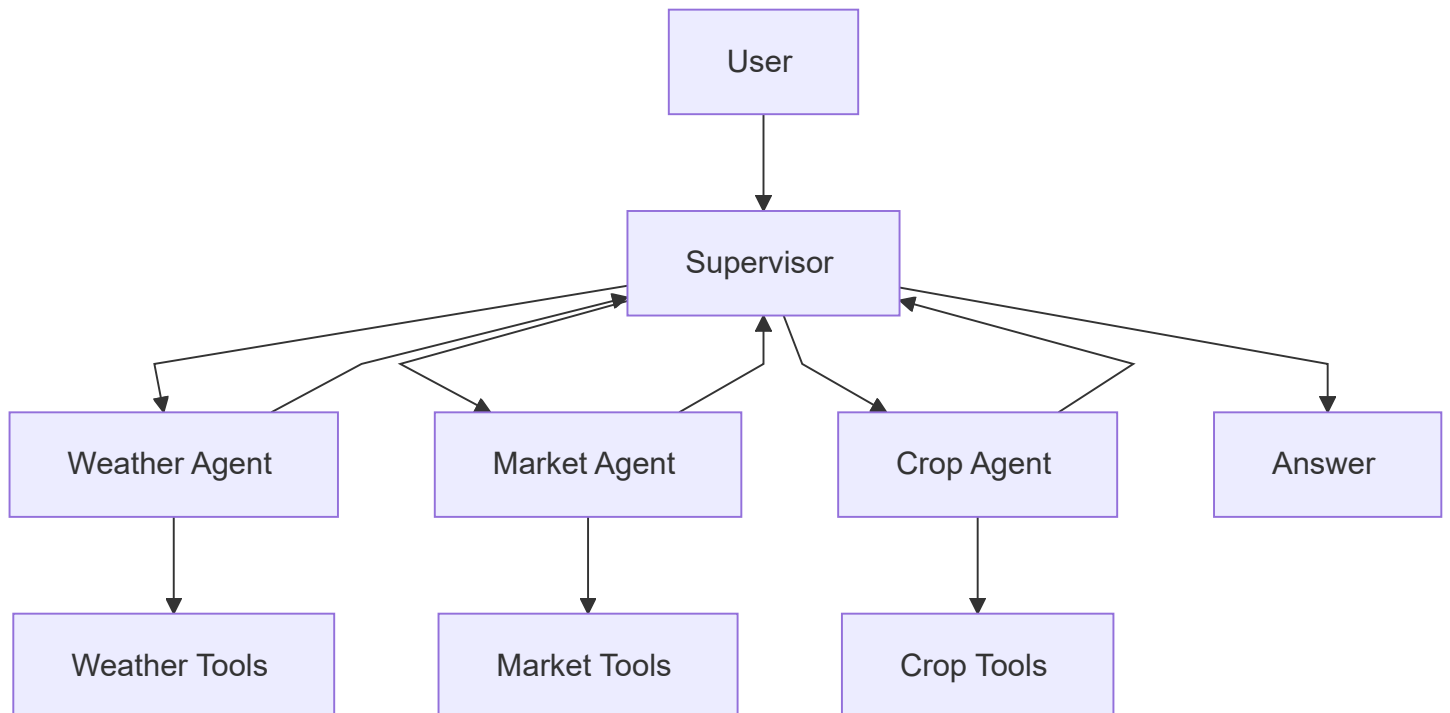
## 4. Single Agent vs Multi-Agent

---

### Single Agent



### Multi-Agent



## 5. When Should We Use Multi-Agent Systems?

---



*Use multiple agents when:*

- Task has clearly different domains
- Different agents need different tools
- Different prompts/expertise are useful
- Workflow has multiple specialized responsibilities
- One agent becomes difficult to maintain
- Agents can work independently on subtasks

*Example:*

- Research Task
  - Search Specialist
  - Data Analyst
  - Report Writer

---

## 6. When NOT to Use Multi-Agent



*Do not create multiple agents just because:*

"Multi-agent sounds advanced."

*If the workflow is:*

User  
↓  
Agent  
↓  
2-3 Tools  
↓  
Answer

*a single agent is usually better.*

*Multi-agent systems add:*

More LLM calls  
  
Higher latency  
  
Higher cost  
  
More state  
  
More routing  
  
More failure points  
  
More debugging

---

# 7. Agent Specialization

---

*The key idea is:*

## Specialization

---

*Instead of:*

General Agent  
→ Everything

*we create:*

Weather Agent  
→ Weather analysis

Market Agent  
→ Market analysis

Crop Agent  
→ Crop recommendation

*Each agent can have its own:*



System Prompt

Tools

Knowledge

State

Rules

---

## 8. Example Specialized Agents

---

### Weather Agent

Responsibility:

Analyze current weather.

Tools:

Weather API

Forecast API

### Crop Agent

Responsibility:

Analyze crop suitability.

Tools:

Crop ML Model

Crop Knowledge Base

## Market Agent

Responsibility:  
Analyze crop prices.

Tools:  
Market Price API  
Historical Price Database

---

## 9. Agent as a Specialized Worker

---

*Conceptually:*

```
weather_agent = create_agent(  
    model=llm,  
    tools=[  
        get_weather,  
        get_forecast  
    ],  
    system_prompt="""  
    You are a weather analysis specialist.  
    Focus only on weather-related tasks.  
    """,  
)
```

*Another agent:*

```
market_agent = create_agent(  
    model=llm,  
    tools=[  
        get_market_price  
    ],  
    system_prompt="""  
    You are a market analysis specialist.  
    Analyze agricultural market data.  
    """,  
)
```

*The exact helper APIs can vary by framework/version; the important concept is specialization.*

---

## 10. Main Multi-Agent Architectures

---



*You should know four important patterns:*

1. Supervisor Architecture
2. Router Architecture
3. Sequential Architecture
4. Parallel Architecture

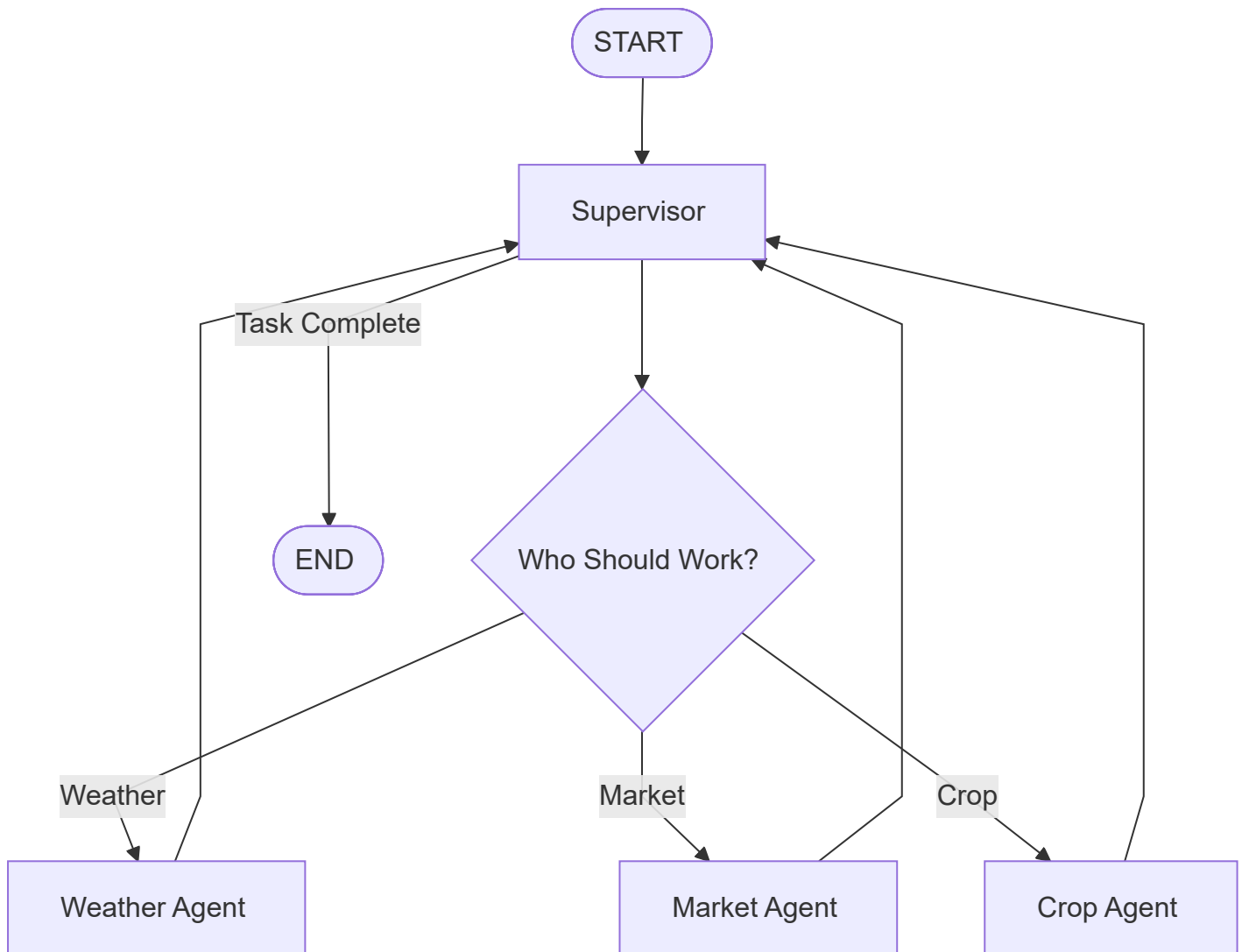
*We will understand each.*

---

## II. Supervisor Architecture



A **supervisor agent** coordinates specialized worker agents.



The supervisor decides:

Which agent should execute next?

Is another agent needed?

Is the task complete?

---

## 12. Supervisor Example

---

*User:*

Should I grow wheat this week?

*Supervisor reasoning:*

Need weather analysis.

↓

Weather Agent

*Result:*

Weather conditions are suitable.

*Supervisor:*

Need crop suitability.

↓

Crop Agent

*Result:*

wheat suitability = high.

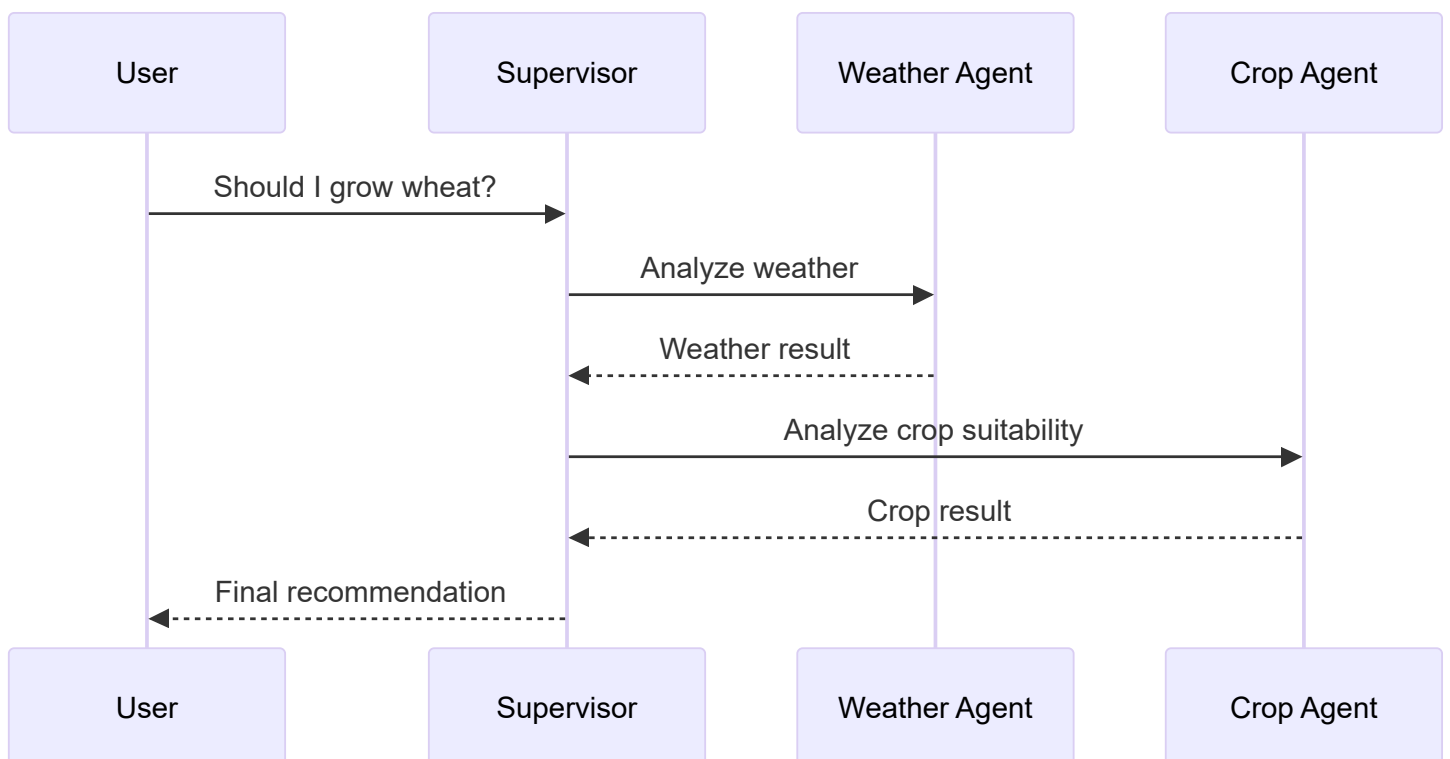
*Supervisor:*

Enough information.



Final Response

## 13. Supervisor Execution



---

## 14. Supervisor as an Agent

---

*The supervisor itself may use an LLM.*

*Its available actions could conceptually be:*

```
weather_agent  
  
crop_agent  
  
market_agent  
  
FINISH
```

*Example structured decision:*

```
from pydantic import BaseModel  
from typing import Literal  
  
class SupervisorDecision(BaseModel):  
  
    next_agent: Literal[  
        "weather",  
        "crop",  
        "market",  
        "finish"  
    ]
```

---

## 15. Supervisor Node

---

*Conceptually:*

```
def supervisor_node(state):  
  
    decision = supervisor.invoke(  
        state["messages"]  
    )  
  
    return {  
        "next_agent":  
            decision.next_agent  
    }
```

*Routing:*

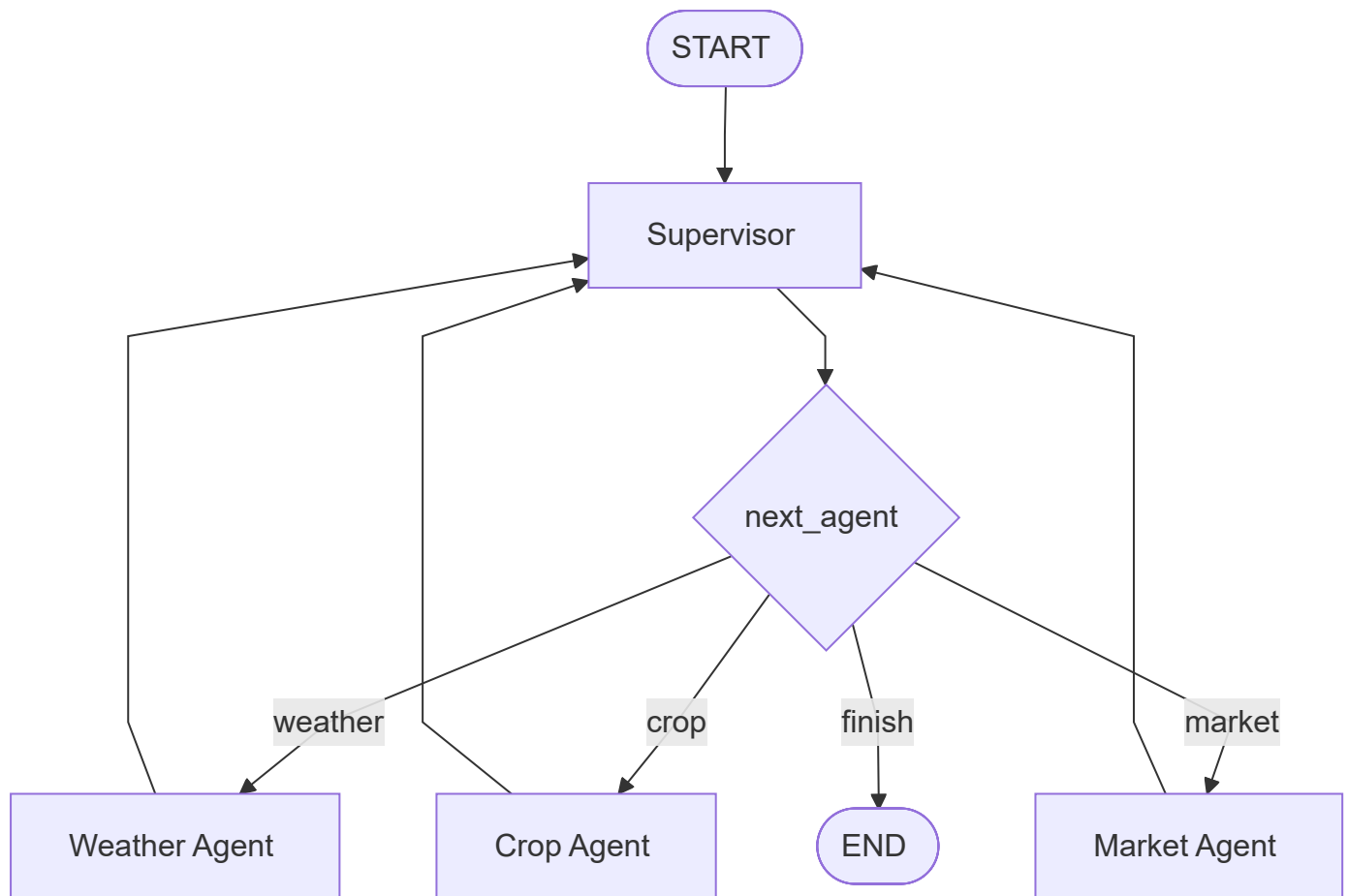
```
def route_supervisor(state):  
  
    return state["next_agent"]
```

---



## 16. Supervisor Graph

---



*This allows dynamic execution.*

---

## 17. Router Architecture

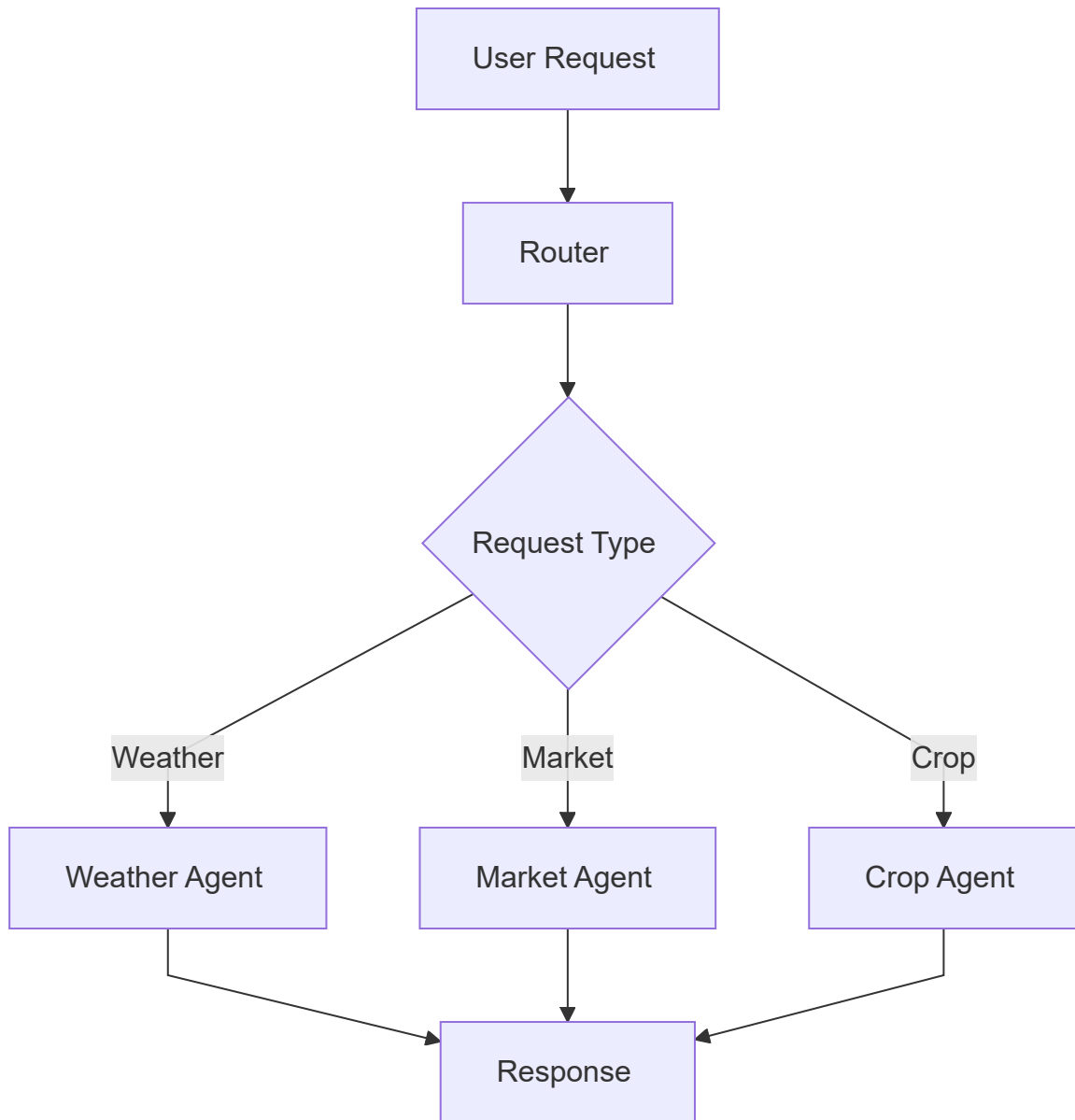
---

*A simpler pattern is a:*

### Router

---

*The router decides which agent should handle the request.*



*Unlike a supervisor, the router may make only one routing decision.*

---

## 18. Router vs Supervisor

---



## Router

Choose one destination

↓

Task handled

↓

Finish

### Example:

"What is today's temperature?"

→ Weather Agent

---

## Supervisor

Choose agent

↓

Observe result

↓

Choose another agent

↓

Observe

↓

Finish

### Example:

"Should I grow wheat this week?"

→ Weather Agent

→ Crop Agent

→ Market Agent

→ Final Answer

## 19. Sequential Architecture

*Some workflows have a fixed order.*

*Example:*

```
Research
↓
Analyze
↓
Write Report
```

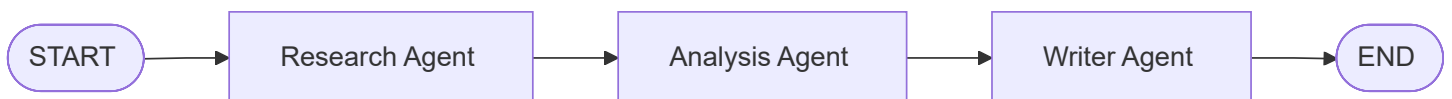
*Agents:*

```
Research Agent

Analysis Agent

Writer Agent
```

*Graph:*



*No supervisor is required if the order is already known.*

## 20. Sequential Example

*User:*

Create a report comparing wheat  
and rice profitability.

*Workflow:*

Research Agent  
→ Collect information

Analysis Agent  
→ Compare data

Writer Agent  
→ Generate report

*Each agent receives the result from the previous step.*

---

## 21. Sequential State

---

*Example:*

```
class ReportState(TypedDict):  
  
    query: str  
  
    research: str  
  
    analysis: str  
  
    report: str
```

*Research node:*

```
return {  
  "research":  
    research_result  
}
```

*Analysis node:*

```
return {  
  "analysis":  
    analysis_result  
}
```

*Writer:*

```
return {  
  "report":  
    final_report  
}
```

---

## 22. Parallel Architecture

---

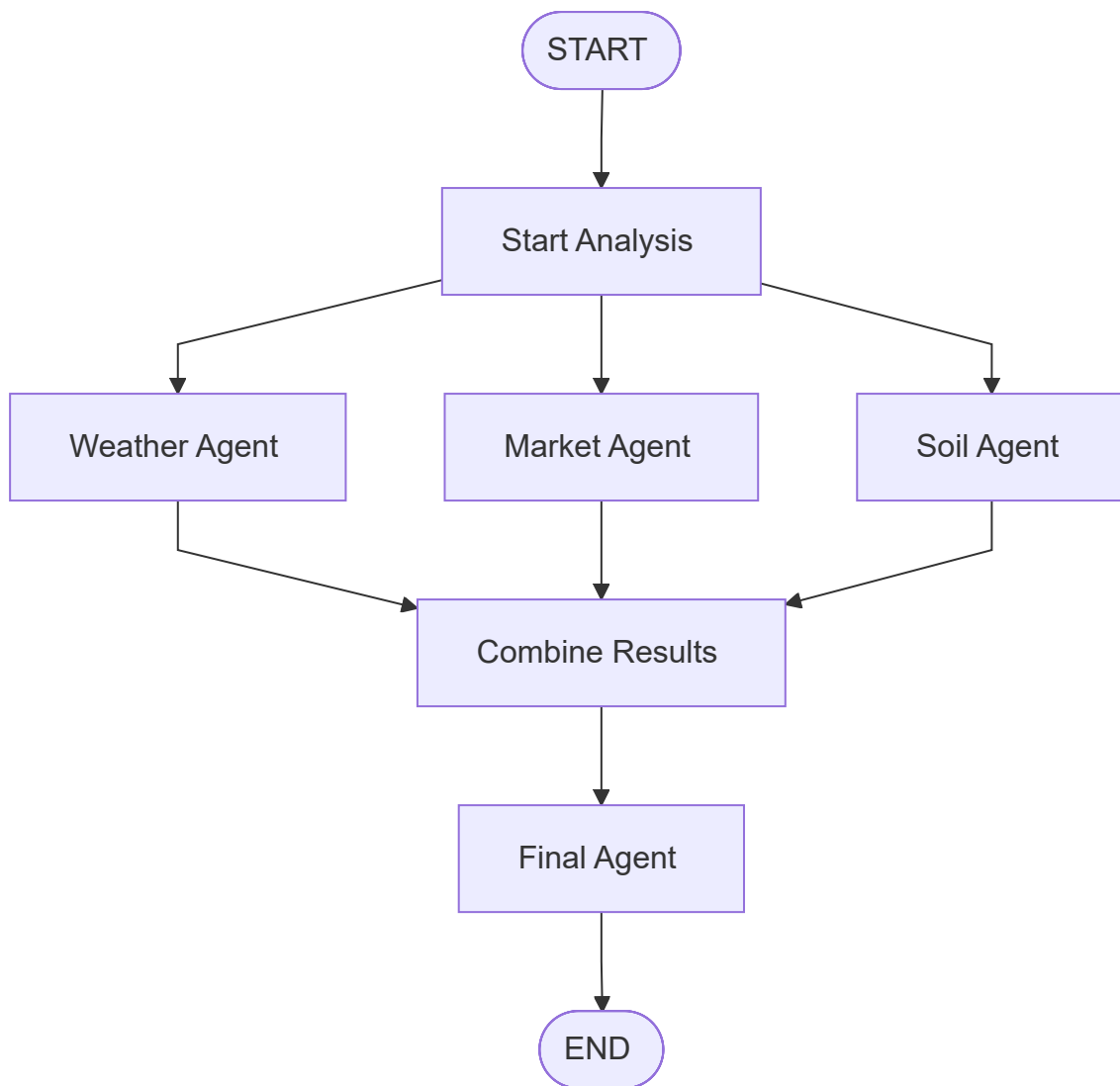


*Some agents can work independently.*

*Example:*

```
Weather Analysis  
  
Market Analysis  
  
Soil Analysis
```

*These may run independently.*



---

## 23. Why Parallel Agents?

---

*Suppose:*

Weather Agent = 2 sec

Market Agent = 3 sec

Soil Agent = 2 sec

*Sequential:*

```
2 + 3 + 2  
≈ 7 seconds
```

*Parallel:*

```
max(2, 3, 2)  
≈ 3 seconds
```

*plus orchestration overhead.*

*This can significantly reduce latency.*

---

## 24. Parallel Agents Need State Merging

---

*Suppose:*

```
Weather Agent  
→ weather_result  
  
Market Agent  
→ market_result  
  
Soil Agent  
→ soil_result
```

*State:*



```
class AnalysisState(TypedDict):
```

```
    query: str
```

```
    weather_result: str
```

```
    market_result: str
```

```
    soil_result: str
```

```
    final_answer: str
```

*Each agent updates a different state field.*

*This makes merging straightforward.*

---

## 25. Shared State

---



*Agents often communicate through shared graph state.*

*Example:*

```
{
    "query":
        "Should I grow wheat?",

    "weather_result":
        "...",

    "market_result":
        "...",

    "crop_result":
```

```
    "...",  
  
    "final_answer":  
        "..."  
}
```

*Agents do not necessarily need to directly message one another.*

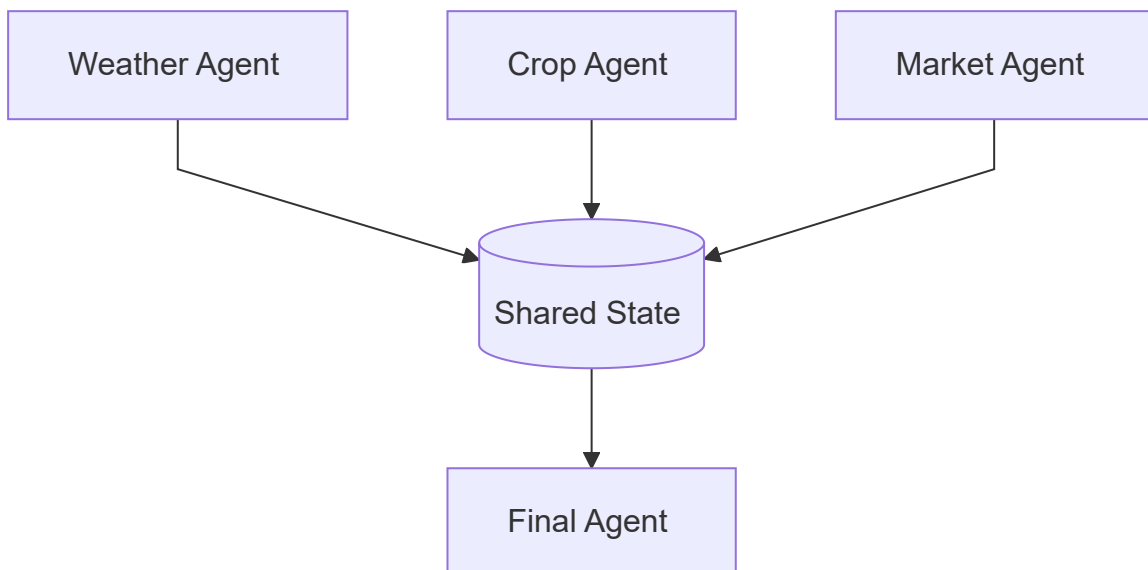
*They can:*

```
Read State  
↓  
Perform Work  
↓  
Update State
```

---

## 26. Shared State Architecture

---



*This can make coordination easier to understand.*

---

## 27. Agent-to-Agent Communication

---

*Another pattern is direct handoff.*

```
Agent A
↓
Agent B
↓
Agent C
```

*For example:*

```
Research Agent
↓
"I collected these facts..."
↓
Analysis Agent
```

*This is useful when the next agent requires the previous agent's output.*

---

## 28. Handoffs

---



A **handoff** transfers responsibility from one agent to another.

*Example:*

General Agent:

"This request requires detailed  
market analysis."

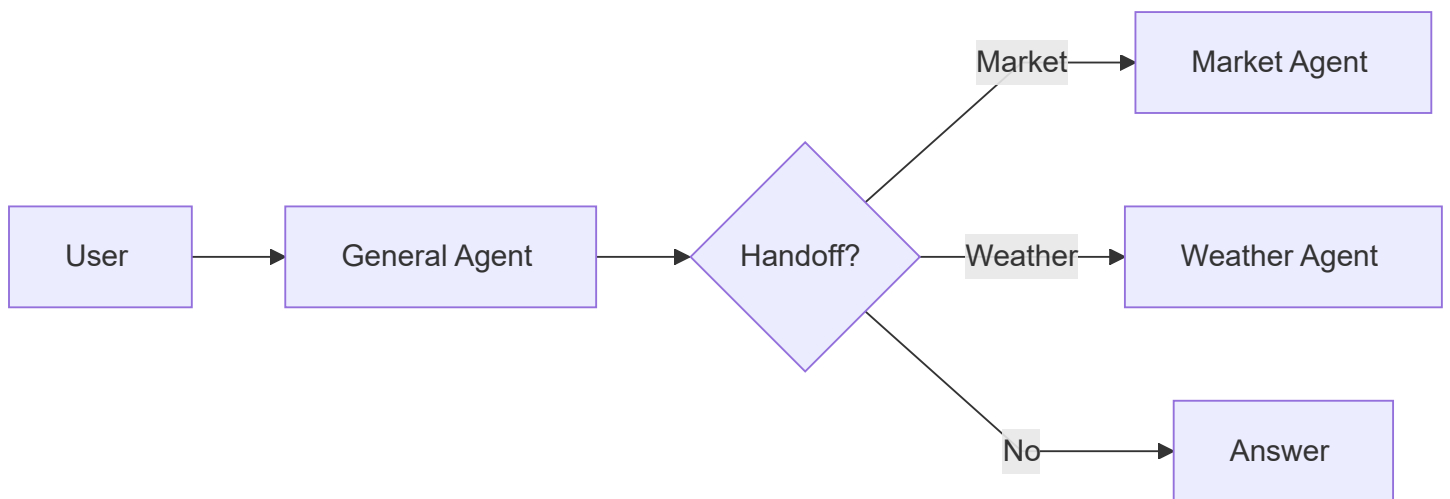
↓

Handoff

↓

Market Agent

*Architecture:*



---

## 29. Handoff vs Tool Call

---

*Important distinction.*

# Tool Call

Agent remains responsible.

Agent

↓

Tool

↓

Result

↓

Same Agent

*Example:*

Crop Agent

→ get\_weather()

→ Crop Agent

---

# Handoff

Responsibility moves to another agent.

Agent A

↓

Agent B

*Example:*

General Agent

→ Market Agent

# 30. When Should Something Be a Tool vs Agent?

---



*This is one of the most important design decisions.*

Use a **tool** when the capability is:

Narrow

Deterministic

Function-like

*Examples:*

`get_weather()`

`search_database()`

`predict_crop()`

`calculate_profit()`

Use an **agent** when the capability requires:

Reasoning

Multiple steps

Its own tools

Its own context

Independent decisions

*Example:*

Market Analysis Agent

- Get current price
- Retrieve historical data
- Compare trend
- Generate market assessment

---

## 31. Bad Multi-Agent Design

---

*Avoid:*

Weather Agent

*whose entire job is:*

```
return get_weather(city)
```

*That should probably just be:*

Weather Tool

*An agent should have enough responsibility to justify another LLM decision loop.*

---

## 32. Good Multi-Agent Design

---

*Example:*

Market Agent

*Responsibilities:*

Determine which market data is needed

Retrieve current prices

Retrieve historical trends

Compare markets

Analyze volatility

Return market assessment

*Now an agent makes more sense.*

---

## 33. Agents Can Have Their Own Tools

---

*Example:*



Market Agent

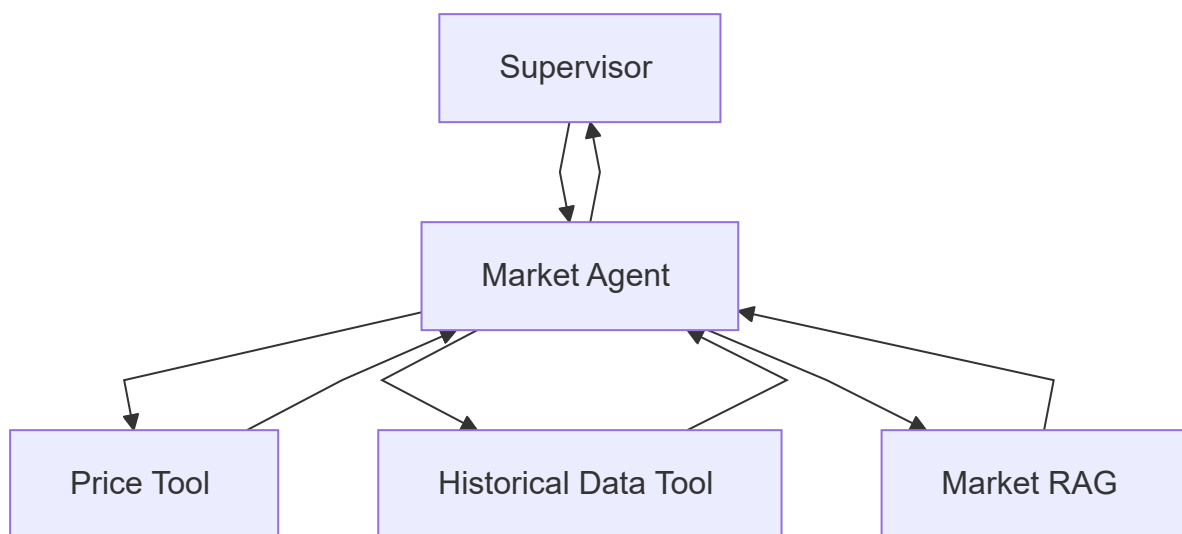
- └ Current Price Tool
- └ Historical Price Tool
- └ Market Search Tool

*Crop Agent:*

Crop Agent

- └ Crop ML Model
- └ Crop RAG
- └ Soil Database

*Architecture:*



---

## 34. Agents Can Use RAG

---

*A specialized agent may have its own knowledge base.*

*Example:*

Crop Agent



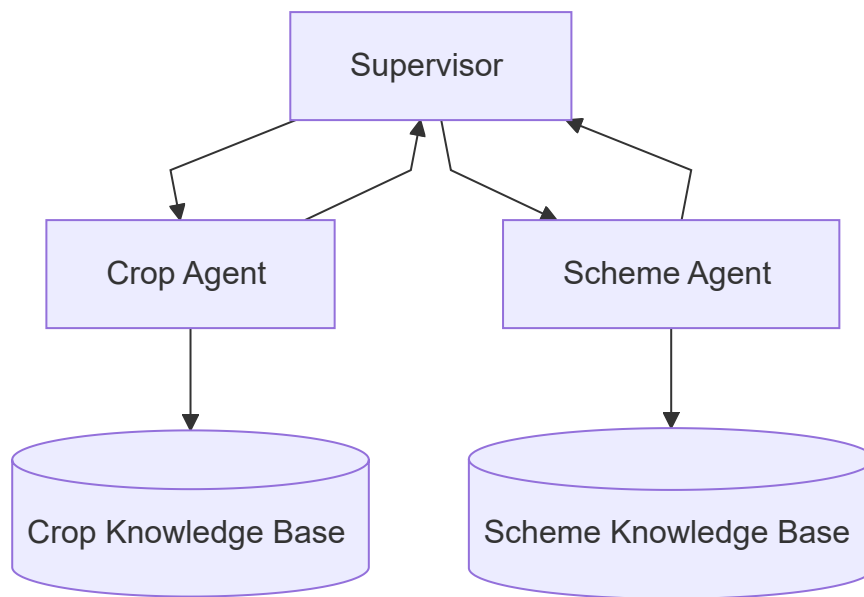
Crop RAG

Scheme Agent



Government Scheme RAG

*Architecture:*



*This combines:*

Multi-Agent

+

Agentic RAG

# 35. Agents Can Use LangGraph Subgraphs



*A specialized agent may itself be a graph.*

*Example:*

```
graph TD;
    MainGraph[Main Graph] --> CropAgent[Crop Agent];
```

*Inside Crop Agent:*

```
graph TD;
    RetrieveSoil[Retrieve Soil] --> GetWeather[Get Weather];
    GetWeather --> RunMLModel[Run ML Model];
    RunMLModel --> Evaluate[Evaluate];
    Evaluate --> ReturnResult[Return Result];
```

*So:*

```
graph TD;
    Agent[Agent];
```

*can sometimes be implemented as a:*

## 36. Hierarchical Architecture

*For larger systems:*

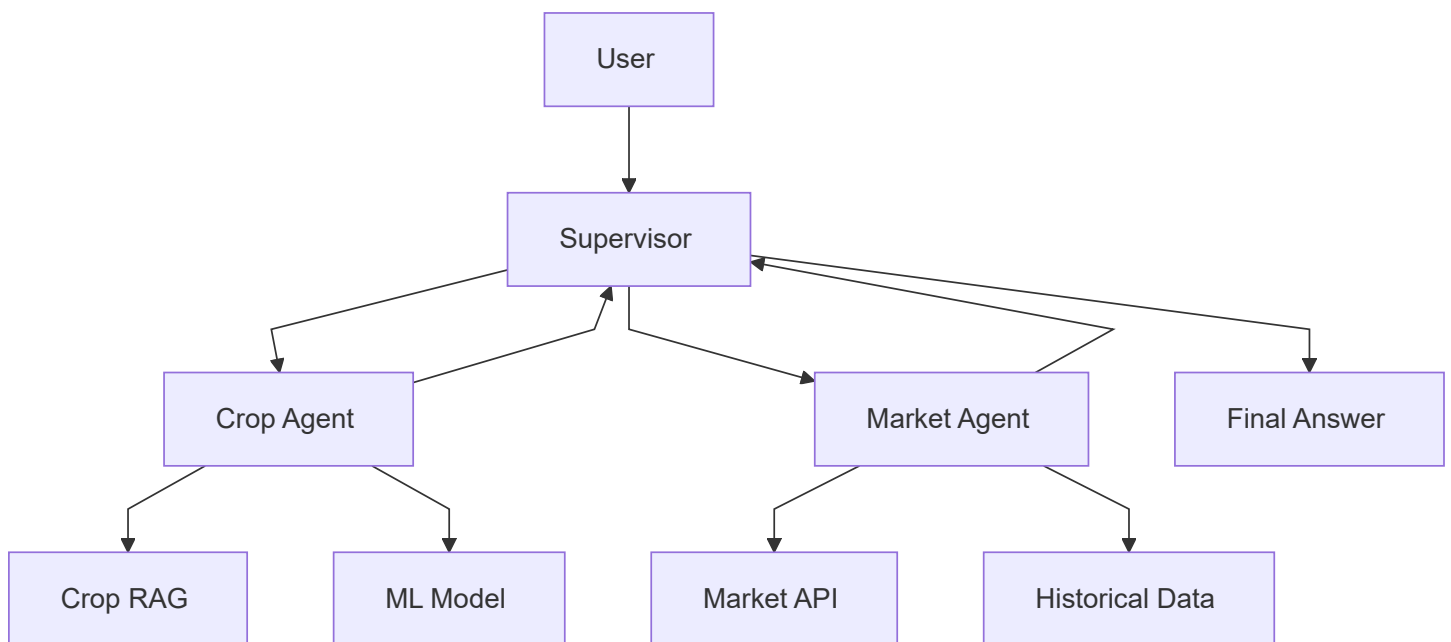
Supervisor



Specialized Agents



Tools / Subgraphs



*This is a hierarchical agent architecture.*

## 37. Final Answer Generation

---

*There are two common approaches.*

### Supervisor Generates Final Answer

```
Worker Results  
↓  
Supervisor  
↓  
Final Answer
```

*Good when supervisor already has enough context.*

---

### Separate Finalizer

```
Worker Results  
↓  
Final Response Agent  
↓  
Answer
```

*Useful when synthesis itself is a substantial task.*

*But avoid creating another agent if a normal LLM call is sufficient.*

---

## 38. Example Finalizer

---

```
def finalizer(state):
```

```
response = llm.invoke(  
    f""  
    User question:  
    {state["query"]}   
  
    weather analysis:  
    {state["weather_result"]}   
  
    Crop analysis:  
    {state["crop_result"]}   
  
    Market analysis:  
    {state["market_result"]}   
  
    Produce a concise final recommendation.  
    ""  
)  
  
return {  
    "final_answer":  
        response.content  
}
```

---

## 39. Avoid Passing Entire Agent Histories

---



*Suppose each agent generates:*

20 messages

*Passing all messages between every agent can rapidly increase context.*

*Instead, pass useful outputs:*

```
Weather Agent
→ Structured Weather Summary

Market Agent
→ Structured Market Summary

Crop Agent
→ Structured Crop Result
```

*Then combine them.*

---

## 40. Structured Agent Output

---

*Example:*

```
class WeatherAnalysis(BaseModel):

    suitable: bool

    temperature: float

    rainfall_risk: str

    summary: str
```

*Weather agent returns:*

```
{  
    "suitable": true,  
    "temperature": 26.5,  
    "rainfall_risk": "low",  
    "summary": "Conditions are suitable."  
}
```

*This is easier for other agents to consume.*

---

## 41. Multi-Agent State

---

*Example:*

```
class MultiAgentState(TypedDict):  
  
    query: str  
  
    next_agent: str  
  
    weather_result: dict  
  
    crop_result: dict  
  
    market_result: dict  
  
    completed_agents: list[str]  
  
    final_answer: str
```

*This gives the supervisor visibility into:*



what has already been done?

what remains?

what information is available?

---

## 42. Prevent Repeated Agent Calls

---

*Suppose supervisor repeatedly chooses:*

Weather Agent

↓

Weather Agent

↓

Weather Agent

*Track:*

completed\_agents

*Example:*

```
[  
  "weather",  
  "crop"  
]
```

*Supervisor prompt can receive:*

Already completed:

weather, crop

*This reduces unnecessary repetition.*

---

## 43. Limit Supervisor Steps

---



*Always define a maximum number of steps.*

```
MAX_STEPS = 8
```

*State:*

```
step_count: int
```

*Then:*

```
if state["step_count"] >= MAX_STEPS:  
  
    return "finalize"
```

*Otherwise multi-agent loops can become expensive or infinite.*

---

## 44. Failure Handling

---

*Suppose:*

```
Weather Agent  
→ Weather API unavailable
```

*Do not crash the whole workflow immediately.*

*The agent can return:*

```
{  
  "status":  
    "failed",  
  
  "reason":  
    "Weather service unavailable"  
}
```

*Supervisor can decide:*

```
Retry  
  
Continue without weather  
  
Ask user  
  
Fallback
```

---

## 45. Human-in-the-Loop with Multi-Agent

---

*Sensitive actions still need approval.*

*Example:*

Supervisor



Market Agent



Proposes Purchase

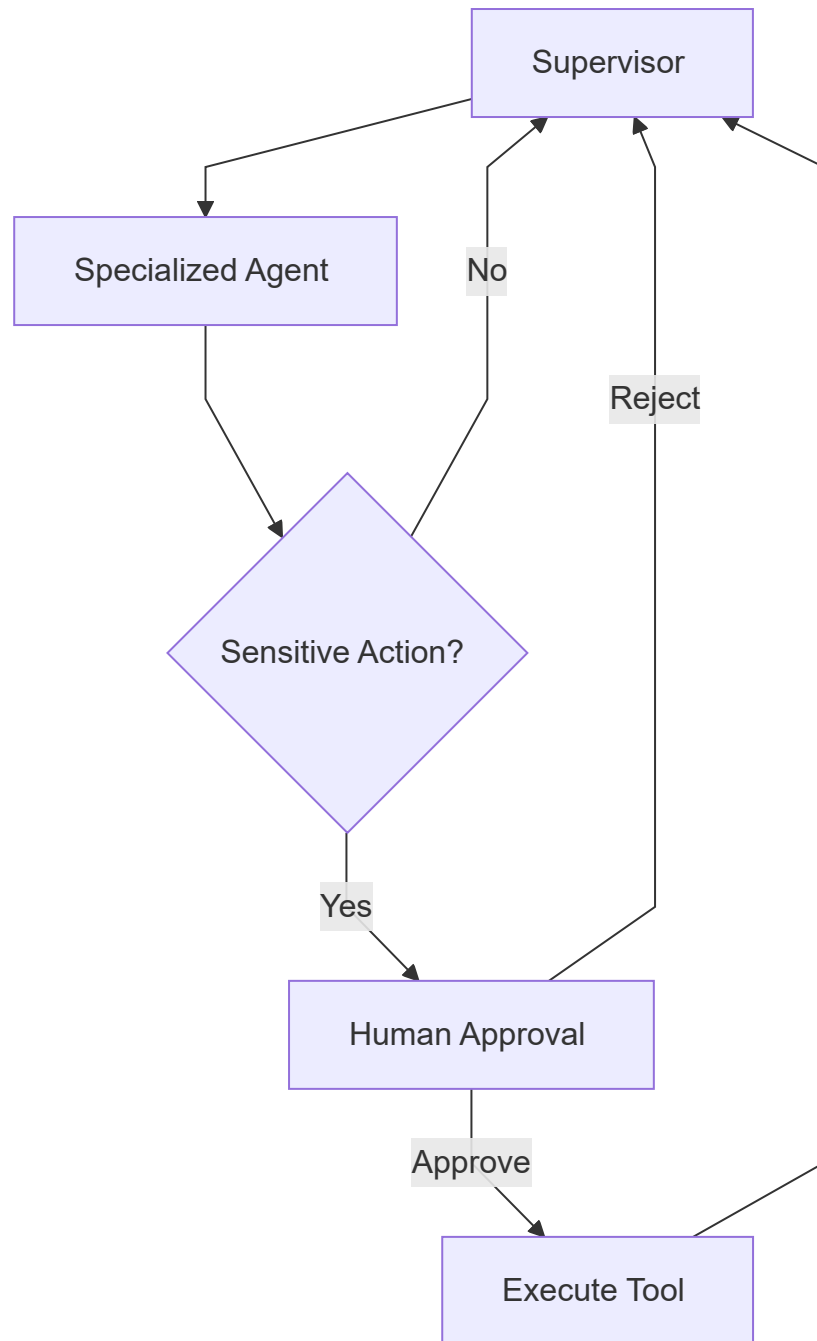


Human Approval



Execute

*Architecture:*



---

## 46. Multi-Agent Does Not Mean Autonomous Everything

---

*Your backend should still control:*

Authentication

Authorization

Tool Permissions

Database Constraints

Validation

Rate Limits

Sensitive Operations

*Agents coordinate reasoning.*

*They should not replace application security.*

---

## 47. Cost of Multi-Agent Systems

---



*Suppose:*

Supervisor = 4 LLM calls

Weather Agent = 2 calls

Crop Agent = 3 calls

Market Agent = 3 calls

Finalizer = 1 call

*Total:*

13 LLM calls

*A single-agent solution might need:*

4 calls

*Therefore:*

*Multi-agent systems should provide enough benefit to justify the extra complexity.*

---

## 48. Multi-Agent Latency

---

*Sequential agents:*

Agent A

2 sec

↓

Agent B

3 sec

↓

Agent C

2 sec

≈ 7 sec

*Parallel agents:*

Agent A  $\neg$   
Agent B  $\neg \rightarrow$  Combine  
Agent C  $\neg$

$\approx$  slowest agent

*Use parallelism when tasks are genuinely independent.*

---

## 49. Debugging Multi-Agent Systems

---

*Track:*

which agent was selected?

why was it selected?

what tools did it call?

what output did it return?

How many times was it called?

How long did it take?

How many tokens were used?

*Without observability, debugging multi-agent systems becomes difficult.*

---



# 50. Common Mistakes

---

## Mistake 1

Turning every function into an agent.

*Use tools for narrow capabilities.*

---

## Mistake 2

Creating agents with overlapping responsibilities.

*Keep responsibilities clear.*

---

## Mistake 3

Unlimited supervisor loops.

*Use step limits.*

---

## Mistake 4

Passing entire histories everywhere.

*Prefer structured summaries/results.*

---

## Mistake 5

Using sequential execution when tasks are independent.

*Consider parallel execution.*

---

## Mistake 6

Using multi-agent when one agent is enough.

*Start simple.*

---

# 51. Choosing an Architecture

---

*Use this mental model:*

One specialized destination needed?

→ Router

Fixed ordered workflow?

→ Sequential

Independent subtasks?

→ Parallel

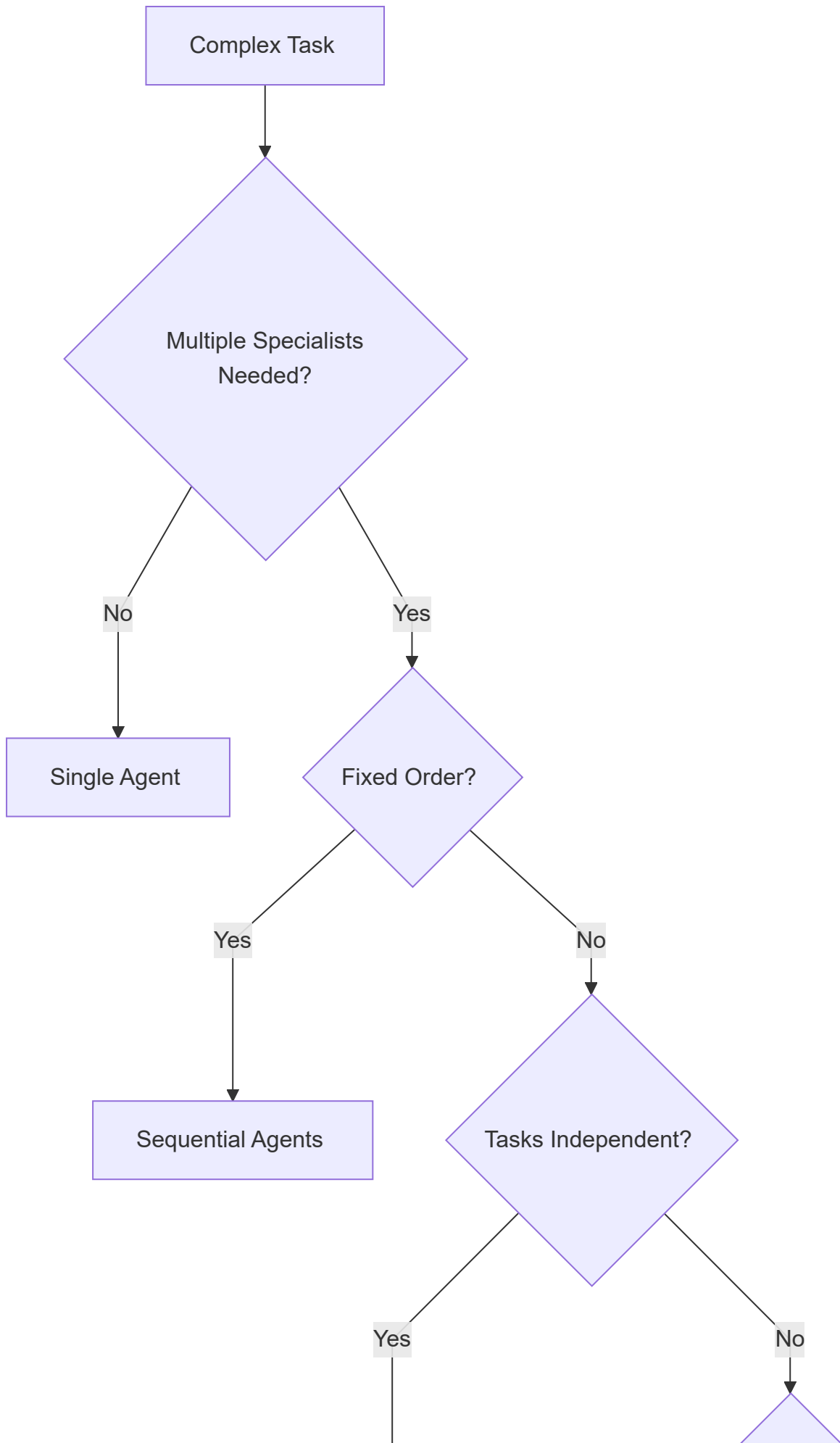
Dynamic multiple-agent coordination?

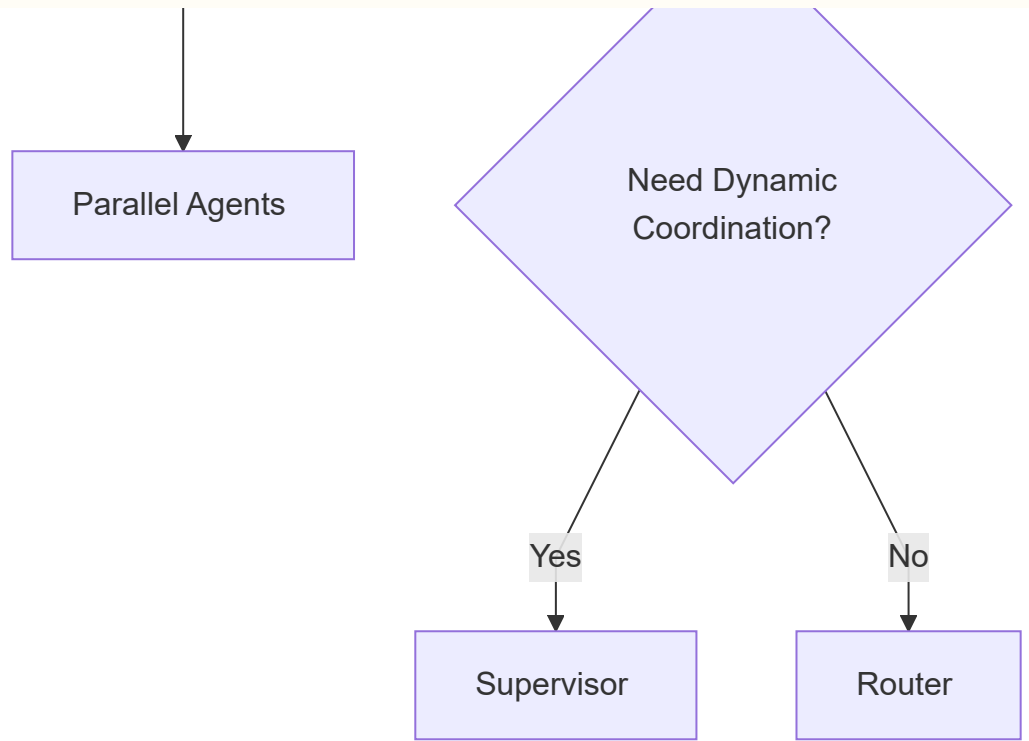
→ Supervisor

---

## *52. Architecture Decision Diagram*

---





---

## 53. Important Interview Questions

---

**Q1. What is a multi-agent system?**



**Answer:**

*A multi-agent system contains multiple specialized agents that coordinate or collaborate to complete tasks, with each agent typically having a focused responsibility, context or toolset.*

---

## Q2. Why use multiple agents instead of one agent? ★ ★ ★ ★ ★

### Answer:

Multiple agents can improve modularity and specialization when a task contains clearly separated domains or complex subtasks. However, they also increase cost, latency and orchestration complexity.

---

## Q3. What is the supervisor pattern? ★ ★ ★ ★ ★

### Answer:

The supervisor pattern uses a central coordinating agent that decides which specialized worker should act next, observes its result and continues routing until the overall task is complete.

---

## Q4. Router vs supervisor? ★ ★ ★ ★ ★

### Answer:

A router typically selects a destination once, while a supervisor can repeatedly select different agents based on intermediate results until the task is complete.

---

## Q5. What is a handoff?

### Answer:

*A handoff transfers responsibility for handling a task from one agent to another specialized agent.*

---

## Q6. Tool call vs handoff? ★ ★ ★ ★ ★

### Answer:

*In a tool call, the same agent remains responsible and uses the tool as a capability. In a handoff, responsibility moves from one agent to another.*

---

## Q7. Tool vs agent? ★ ★ ★ ★ ★

### Answer:

*Use a tool for narrow function-like capabilities. Use an agent when the task requires reasoning, multiple steps, its own tools or independent decision-making.*

---

## Q8. What is a sequential multi-agent architecture?

### Answer:

*Agents execute in a predefined order, with the output of one agent becoming input to the next.*

---

## Q9. What is parallel multi-agent execution?

### Answer:

*Independent agents execute concurrently and their results are later combined, reducing latency when their tasks do not depend on each other.*

---

## Q10. Can agents have their own RAG systems?

### Answer:

*Yes. Specialized agents can have dedicated retrievers or knowledge bases relevant to their domain.*

---

## Q11. Can an agent be implemented as a LangGraph subgraph?

### Answer:

*Yes. A specialized agent may internally contain a multi-step graph and be integrated into a larger parent workflow as a subgraph.*

---

## Q12. What are the disadvantages of multi-agent systems?

### Answer:



They can increase LLM calls, token usage, latency, state-management complexity, routing errors, debugging difficulty and overall system cost.

## 54. Final Mental Model

Start with:

```
graph TD
    A[Single Agent] --> B[Tools]
```

When responsibilities become genuinely separate:

```
graph TD
    S[Supervisor] --> W[Weather Agent]
    S --> C[Crop Agent]
    S --> M[Market Agent]
    W --> T1[Tools]
    C --> T2[Tools]
    M --> T3[Tools]
```

The main patterns are:

```
graph TD
    R[ROUTER] --> Q[Query]
    Q --> C[Choose Agent]
    C --> A[Agent]
    A --> Ans[Answer]
```

## SEQUENTIAL

Agent A

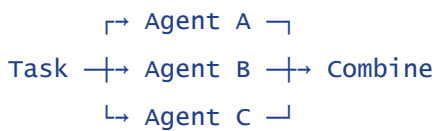


Agent B



Agent C

## PARALLEL



## SUPERVISOR

Supervisor



Worker



Supervisor



Another worker



Supervisor



Finish

*And remember the most important design rule:*

*Do not create another agent when a tool, normal function, deterministic node or simple LLM call can solve the responsibility cleanly. Multi-agent architecture is useful when specialization and coordination provide a real engineering benefit.*

# Part 2 – Implementing Multi-Agent Systems with LangGraph

---

---

## 1. What Are We Building?

---

*Now we will implement the supervisor architecture from Part 1.*

*We will build:*

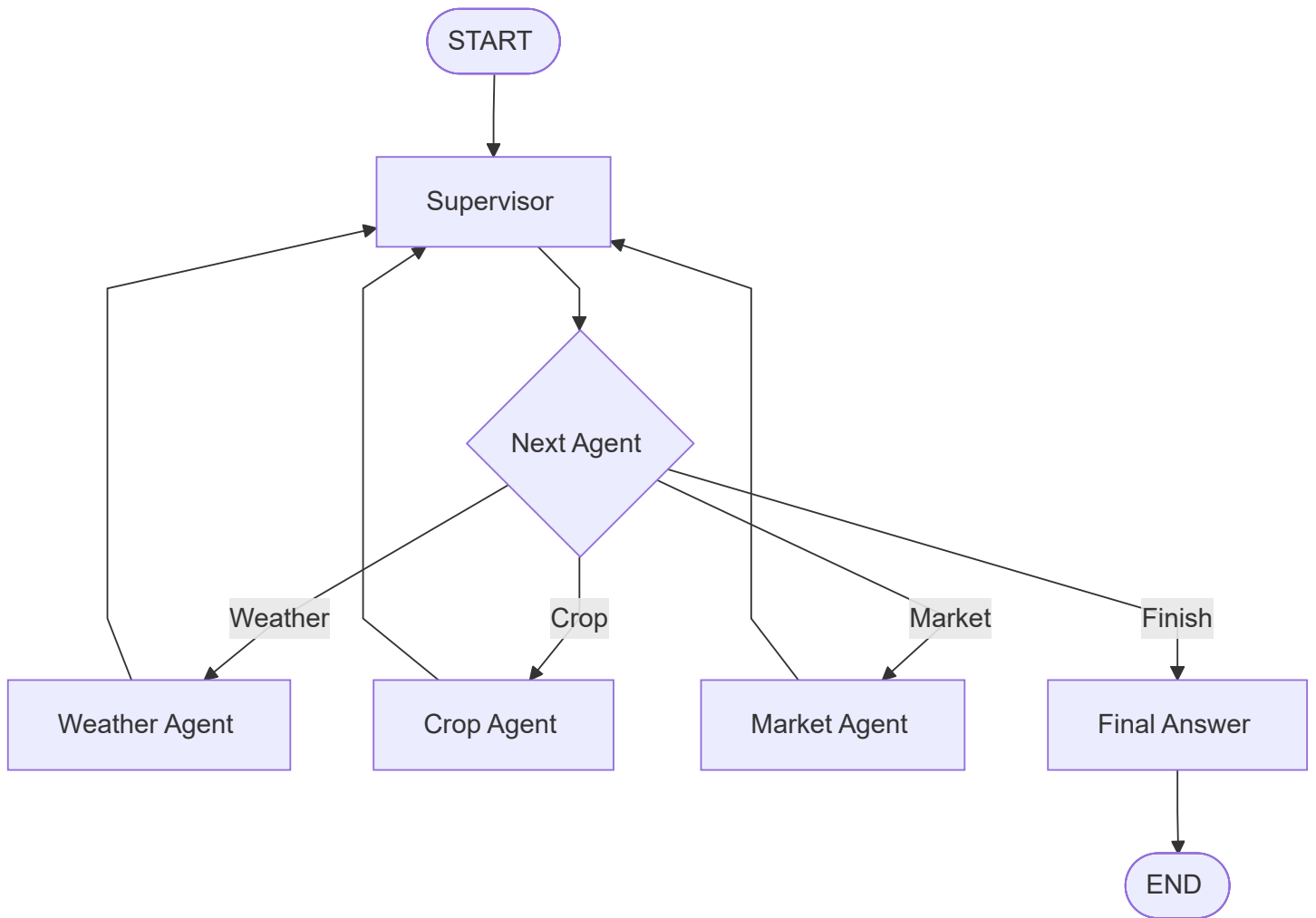
Supervisor

└─ Weather Agent

└─ Crop Agent

└─ Market Agent

*The supervisor decides which specialist should work next.*



---

## 2. Example User Request

---

*Suppose the user asks:*

Should I grow wheat in Gorakhpur  
this season?

*The supervisor may decide:*

Step 1  
→ Weather Agent

Step 2  
→ Crop Agent

Step 3  
→ Market Agent

Step 4  
→ Final Answer

*But this sequence is **not hardcoded**.*

*For another query:*

what is the current weather in Gorakhpur?

*the flow may simply be:*

Supervisor  
↓  
Weather Agent  
↓  
Supervisor  
↓  
Finish

---

## 3. Define Shared State

---



*All agents need some shared information.*

```
from typing import TypedDict

class AgentState(TypedDict):

    query: str

    next_agent: str

    weather_result: str

    crop_result: str

    market_result: str

    completed_agents: list[str]

    step_count: int

    final_answer: str
```

---

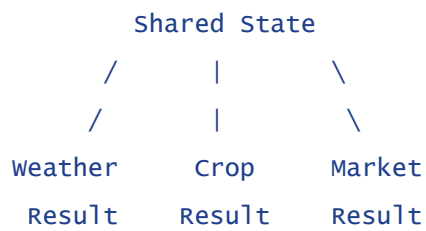
## 4. Why Shared State?

---

*Instead of agents directly sending large conversations to one another:*

```
Weather Agent
↓
Crop Agent
↓
Market Agent
```

*we maintain:*



*Each worker:*

Reads required state

Performs its task

Writes structured result

## 5. Initial State

```
initial_state = {  
  
    "query":  
        "Should I grow wheat in Gorakhpur?",  
  
    "next_agent":  
        "",  
  
    "weather_result":  
        "",  
  
    "crop_result":  
        "",  
  
    "market_result":  
        "",  
  
    "completed_agents":  
        [],  
}
```

```
"step_count":  
    0,  
  
"final_answer":  
    ""  
}
```

---

## 6. Supervisor Decision Schema

---



*We should not ask the supervisor to return free-form text such as:*

```
I think we should probably  
call the weather specialist.
```

*Use structured output.*

```
from pydantic import BaseModel  
from typing import Literal  
  
class SupervisorDecision(BaseModel):  
  
    next_agent: Literal[  
        "weather",  
        "crop",  
        "market",  
        "finish"  
    ]
```



## 7. Create Supervisor

---

```
supervisor = (  
    llm.with_structured_output(  
        SupervisorDecision  
    )  
)
```

*The supervisor now returns something like:*

```
SupervisorDecision(  
    next_agent="weather"  
)
```

*rather than arbitrary text.*

---

## 8. Supervisor Prompt

---

*The supervisor should know:*

```
original Query  
  
Available Specialists  
  
Already Completed work  
  
Current Results
```

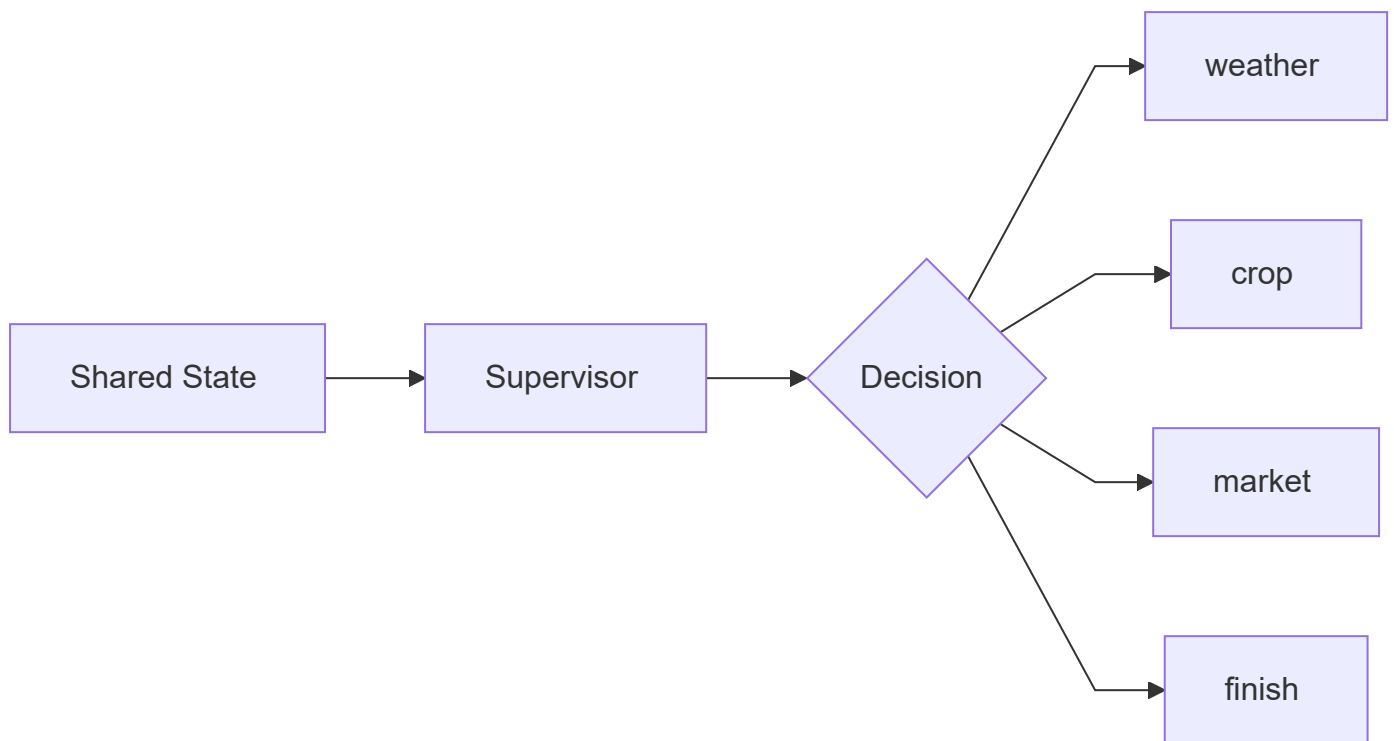
*Example:*

```
def supervisor_node(state):
```

```
result = supervisor.invoke(  
    f"""  
    You are a supervisor coordinating  
    specialized agricultural agents.  
  
    User request:  
    {state["query"]}  
  
    Available agents:  
  
    weather:  
    analyzes weather conditions.  
  
    crop:  
    analyzes crop suitability.  
  
    market:  
    analyzes market conditions.  
  
    Completed agents:  
    {state["completed_agents"]}  
  
    Weather result:  
    {state["weather_result"]}  
  
    Crop result:  
    {state["crop_result"]}  
  
    Market result:  
    {state["market_result"]}  
  
    Select the next required agent.  
  
    If enough information is available,  
    choose finish.  
  
    Do not unnecessarily repeat  
    completed agents.  
    """,  
    )
```

```
return {  
  "next_agent":  
    result.next_agent,  
  
  "step_count":  
    state["step_count"] + 1  
}
```

## 9. Supervisor Flow



*The supervisor does not necessarily perform domain analysis itself.*

*Its primary responsibility is:*

Coordination

# 10. Weather Agent

---

*The weather agent may have access to:*

Current Weather Tool

Forecast Tool

*Conceptually:*

```
def weather_agent_node(state):

    weather = get_weather(
        location="Gorakhpur"
    )

    response = llm.invoke(
        f"""
        User request:
        {state["query"]}

        Weather data:
        {weather}

        Analyze only the weather-related
        implications of this request.
        """
    )

    return {
        "weather_result":
            response.content,

        "completed_agents":
            state["completed_agents"]
            + ["weather"]
    }
```

```
}
```

---

## 11. Important Improvement

---

*In a real application, do not hardcode:*

```
location="Gorakhpur"
```

*The location should come from:*

```
User Request
```

```
Validated Application State
```

```
User Input
```

```
Database
```

*For example:*

```
location = state["location"]
```

---

## 12. Better State

---

*We could extend the state:*

```
class AgentState(TypedDict):
```

```
    query: str
```

```
location: str

crop: str

next_agent: str

weather_result: str

crop_result: str

market_result: str

completed_agents: list[str]

step_count: int

final_answer: str
```

*Now tools receive structured inputs.*

---

## 13. Crop Agent

---

*Suppose we already have a crop prediction model exposed through a tool:*

```
def crop_agent_node(state):

    prediction = predict_crop(
        location=state["location"],
        crop=state["crop"]
    )

    response = llm.invoke(
        f"""
        User request:
        {state["query"]}

        Crop prediction:
```

```

        {prediction}

    weather analysis:
    {state["weather_result"]}

    Analyze crop suitability.
    """
)

return {
    "crop_result":
        response.content,

    "completed_agents":
        state["completed_agents"]
        + ["crop"]
}

```

*Notice:*

Crop Agent

*can use:*

Weather Result

*if available.*

## 14. Market Agent

```

def market_agent_node(state):

    market_data = get_market_price(
        crop=state["crop"],

```

```
        location=state["location"]
    )

    response = llm.invoke(
        f"""
        User request:
        {state["query"]}

        Market data:
        {market_data}

        Analyze the market situation
        for the requested crop.
        """
    )

    return {
        "market_result":
            response.content,

        "completed_agents":
            state["completed_agents"]
            + ["market"]
    }
```

---

## 15. Worker Pattern

---

*All workers follow approximately:*



Read State



Use Specialized Tools



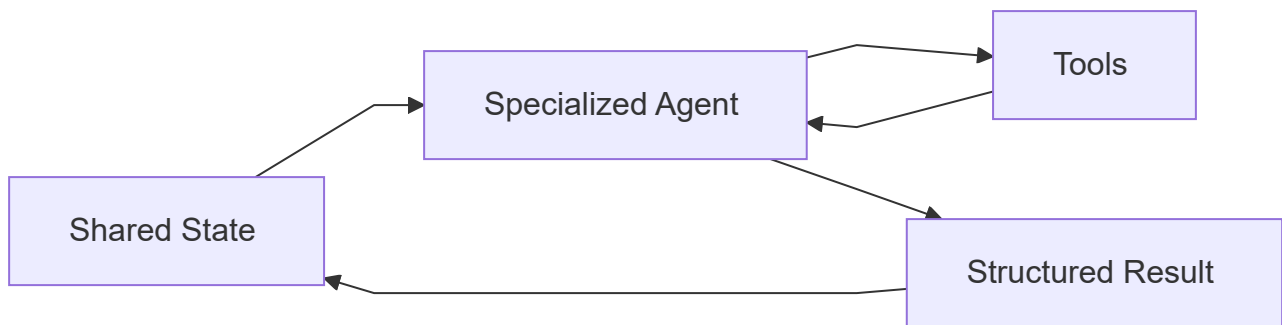
Analyze Result



Update State



Return to Supervisor



---

## 16. Route Supervisor Decision

---

```
MAX_STEPS = 8
```

```
def route_supervisor(state):
```

```
    if state["step_count"] >= MAX_STEPS:
```

```
        return "finish"
```

```
    return state["next_agent"]
```

*Possible outputs:*

weather

crop

market

finish

---

## 17. Why MAX\_STEPS?

---



*Without a limit:*

Supervisor

↓

weather

↓

Supervisor

↓

Crop

↓

Supervisor

↓

weather

↓

Supervisor

↓

weather

↓

...

*could continue unnecessarily.*

*Always provide:*

Termination Condition

*for agent loops.*

---

## 18. Finalizer Node

---

*Once the supervisor selects:*

finish

*we combine the worker results.*

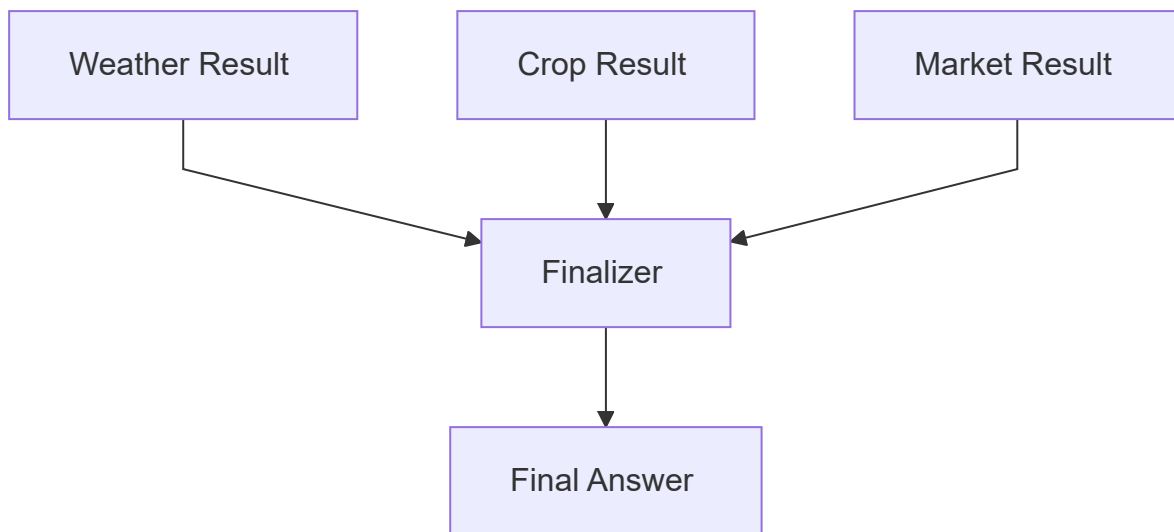
```
def finalizer_node(state):  
  
    response = llm.invoke(  
        f"""  
        User request:  
        {state["query"]}  
  
        weather analysis:  
        {state["weather_result"]}  
  
        Crop analysis:  
        {state["crop_result"]}  
  
        Market analysis:  
        {state["market_result"]}  
  
        Produce a concise final  
        recommendation.  
  
        Do not invent missing data.  
        """)  
  
    )
```

```
return {  
    "final_answer":  
        response.content  
}
```

---

## 19. Final Synthesis

---



*The finalizer should work from useful worker outputs rather than repeating all tool calls.*

---

## 20. Build LangGraph

---

*Imports:*

```
from langgraph.graph import (  
    StateGraph,  
    START,  
    END  
)
```

Create graph:

```
builder = StateGraph(  
    AgentState  
)
```

## 21. Add Nodes

```
builder.add_node(  
    "supervisor",  
    supervisor_node  
)  
  
builder.add_node(  
    "weather",  
    weather_agent_node  
)  
  
builder.add_node(  
    "crop",  
    crop_agent_node  
)  
  
builder.add_node(  
    "market",  
    market_agent_node  
)  
  
builder.add_node(  
    "finish",  
    finalizer_node  
)
```

## 22. START Edge

---

```
builder.add_edge(  
    START,  
    "supervisor"  
)
```

*So every request begins with:*

```
Supervisor
```

---

## 23. Conditional Routing

---

```
builder.add_conditional_edges(  
    "supervisor",  
    route_supervisor,  
    {  
        "weather":  
            "weather",  
  
        "crop":  
            "crop",  
  
        "market":  
            "market",  
  
        "finish":  
            "finish"  
    }  
)
```

*This is the main multi-agent routing logic.*

## 24. Workers Return to Supervisor

```
builder.add_edge(  
    "weather",  
    "supervisor"  
)  
  
builder.add_edge(  
    "crop",  
    "supervisor"  
)  
  
builder.add_edge(  
    "market",  
    "supervisor"  
)
```

*Architecture:*

```
Supervisor  
↓  
Worker  
↓  
Supervisor
```

*This creates the coordination loop.*

## 25. Finish Edge

---

```
builder.add_edge(  
    "finish",  
    END  
)
```

---

## 26. Compile

---

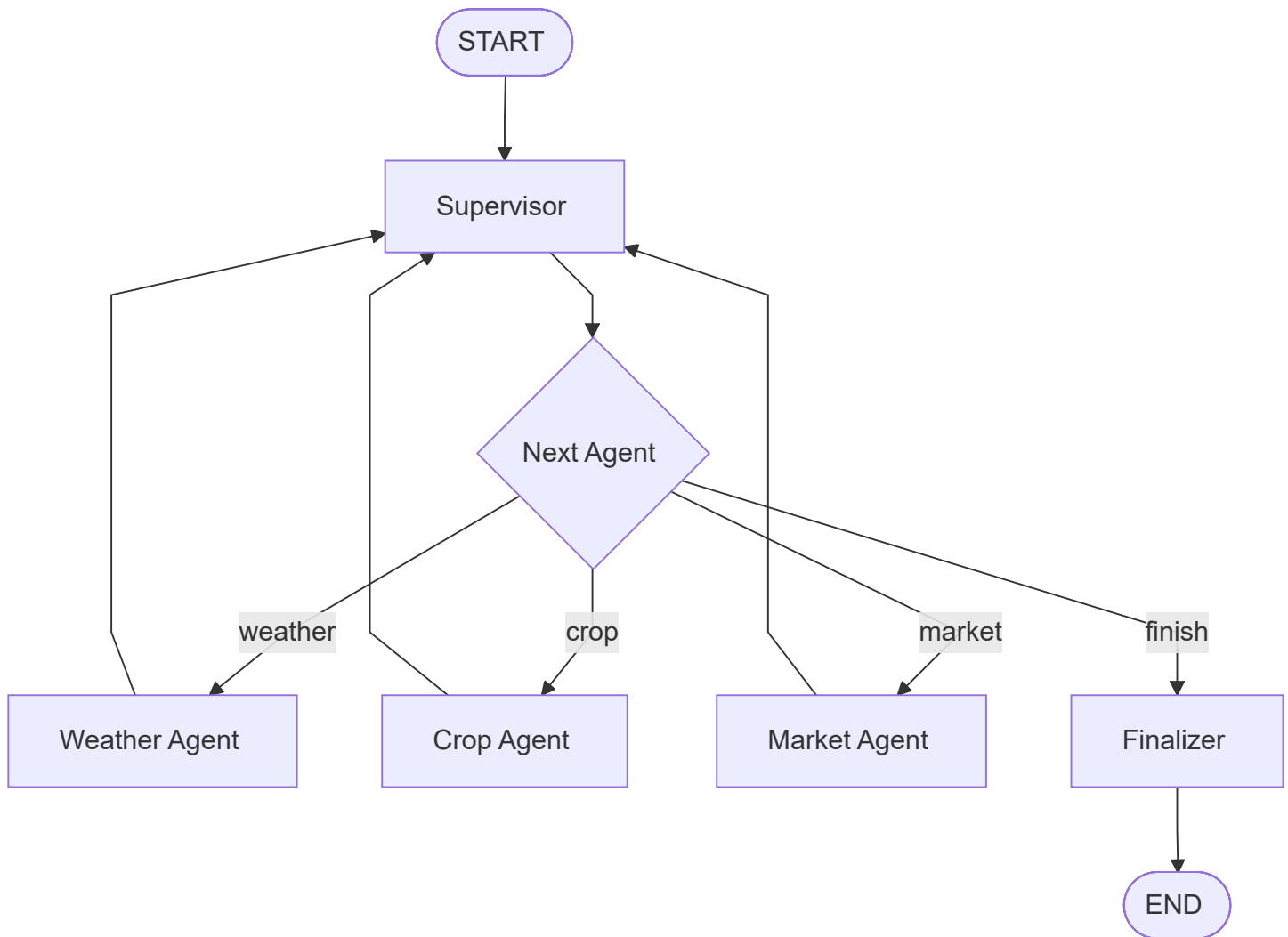
```
graph = builder.compile()
```

*We now have a basic supervisor multi-agent system.*

---



## 27. Complete Graph



## 28. Core Implementation Together



```
from typing import TypedDict, Literal
from pydantic import BaseModel

from langgraph.graph import (
    StateGraph,
```

```
START,  
END  
)
```

```
# -----  
# State  
# -----
```

```
class AgentState(TypedDict):
```

```
    query: str
```

```
    location: str
```

```
    crop: str
```

```
    next_agent: str
```

```
    weather_result: str
```

```
    crop_result: str
```

```
    market_result: str
```

```
    completed_agents: list[str]
```

```
    step_count: int
```

```
    final_answer: str
```

```
# -----  
# Supervisor Schema  
# -----
```

```
class SupervisorDecision(BaseModel):
```

```
    next_agent: Literal[  
        "weather",  
        "crop",
```

```
        "market",
        "finish"
    ]
```

```
supervisor = (
    llm.with_structured_output(
        SupervisorDecision
    )
)
```

```
MAX_STEPS = 8
```

```
# -----
# Supervisor
# -----
```

```
def supervisor_node(state):
```

```
    decision = supervisor.invoke(
        f"""
        Coordinate these agents:

        weather
        crop
        market

        User request:
        {state["query"]}

        Completed:
        {state["completed_agents"]}

        weather:
        {state["weather_result"]}

        Crop:
        {state["crop_result"]}
    """
    )
```

Market:

```
{state["market_result"]}
```

Select the next required agent.

Avoid unnecessary repetition.

Select finish when enough  
information is available.  
"""

```
)
```

```
return {
```

```
    "next_agent":
```

```
        decision.next_agent,
```

```
    "step_count":
```

```
        state["step_count"] + 1
```

```
}
```

```
def route_supervisor(state):
```

```
    if state["step_count"] >= MAX_STEPS:
```

```
        return "finish"
```

```
    return state["next_agent"]
```

```
# -----
```

```
# Weather Worker
```

```
# -----
```

```
def weather_agent_node(state):
```

```
    weather = get_weather(
```

```
        state["location"]
```

```
)
```

```
    result = llm.invoke(
```

```

        f"""
        User request:
        {state["query"]}

        Weather:
        {weather}

        Give only the weather-related
        analysis needed for the request.
        """
    )

```

```

    return {
        "weather_result":
            result.content,

        "completed_agents":
            state["completed_agents"]
            + ["weather"]
    }

```

```

# -----
# Crop worker
# -----

```

```

def crop_agent_node(state):

    prediction = predict_crop(
        location=state["location"],
        crop=state["crop"]
    )

    result = llm.invoke(
        f"""
        User request:
        {state["query"]}

        Prediction:
        {prediction}

```

```

        weather analysis:
        {state["weather_result"]}

        Analyze crop suitability.
        """
    )

    return {
        "crop_result":
            result.content,

        "completed_agents":
            state["completed_agents"]
            + ["crop"]
    }

```

```

# -----
# Market Worker
# -----

```

```

def market_agent_node(state):

    market = get_market_price(
        crop=state["crop"],
        location=state["location"]
    )

    result = llm.invoke(
        f"""
        User request:
        {state["query"]}

        Market data:
        {market}

        Analyze the market situation.
        """
    )

    return {

```

```
        "market_result":
            result.content,

        "completed_agents":
            state["completed_agents"]
            + ["market"]
    }
```

```
# -----
# Finalizer
# -----
```

```
def finalizer_node(state):
```

```
    result = llm.invoke(
        f"""
        User request:
        {state["query"]}

        Weather analysis:
        {state["weather_result"]}

        Crop analysis:
        {state["crop_result"]}

        Market analysis:
        {state["market_result"]}

        Produce the final recommendation.

        Do not invent missing information.
        """
    )

    return {
        "final_answer":
            result.content
    }
```

```
# -----  
# Graph  
# -----
```

```
builder = StateGraph(  
    AgentState  
)
```

```
builder.add_node(  
    "supervisor",  
    supervisor_node  
)
```

```
builder.add_node(  
    "weather",  
    weather_agent_node  
)
```

```
builder.add_node(  
    "crop",  
    crop_agent_node  
)
```

```
builder.add_node(  
    "market",  
    market_agent_node  
)
```

```
builder.add_node(  
    "finish",  
    finalizer_node  
)
```

```
builder.add_edge(  
    START,  
    "supervisor"  
)
```



```
builder.add_conditional_edges(  
    "supervisor",  
    route_supervisor,  
    {  
        "weather": "weather",  
        "crop": "crop",  
        "market": "market",  
        "finish": "finish"  
    }  
)
```

```
builder.add_edge(  
    "weather",  
    "supervisor"  
)
```

```
builder.add_edge(  
    "crop",  
    "supervisor"  
)
```

```
builder.add_edge(  
    "market",  
    "supervisor"  
)
```

```
builder.add_edge(  
    "finish",  
    END  
)
```

```
graph = builder.compile()
```

---

## 29. Invoke the System

---

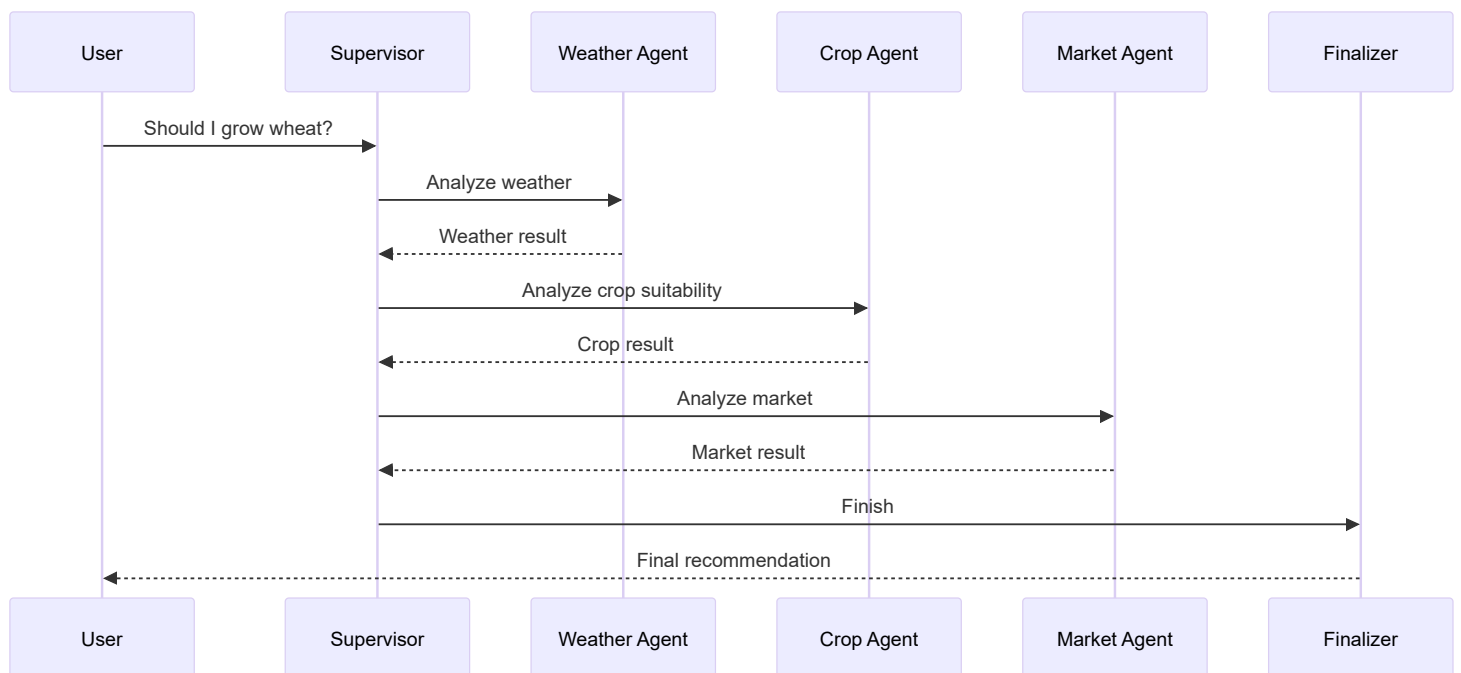
```
result = graph.invoke(  
    {  
        "query":  
            "Should I grow wheat in Gorakhpur?",  
  
        "location":  
            "Gorakhpur",  
  
        "crop":  
            "wheat",  
  
        "next_agent":  
            "",  
  
        "weather_result":  
            "",  
  
        "crop_result":  
            "",  
  
        "market_result":  
            "",  
  
        "completed_agents":  
            [],  
  
        "step_count":  
            0,  
  
        "final_answer":  
            ""  
    }  
)
```

*Then:*

```
print(  
    result["final_answer"]  
)
```

## 30. Example Execution

*Possible execution:*



## 31. But There Is a Problem

*Our workers currently contain logic such as:*

```
weather = get_weather(...)
```

*followed by:*

```
llm.invoke(...)
```

*This is fine when the worker has a predictable workflow.*

*But what if the Weather Agent itself needs to decide:*

```
Current weather?
```

```
7-day forecast?
```

```
Rainfall history?
```

```
Multiple tools?
```

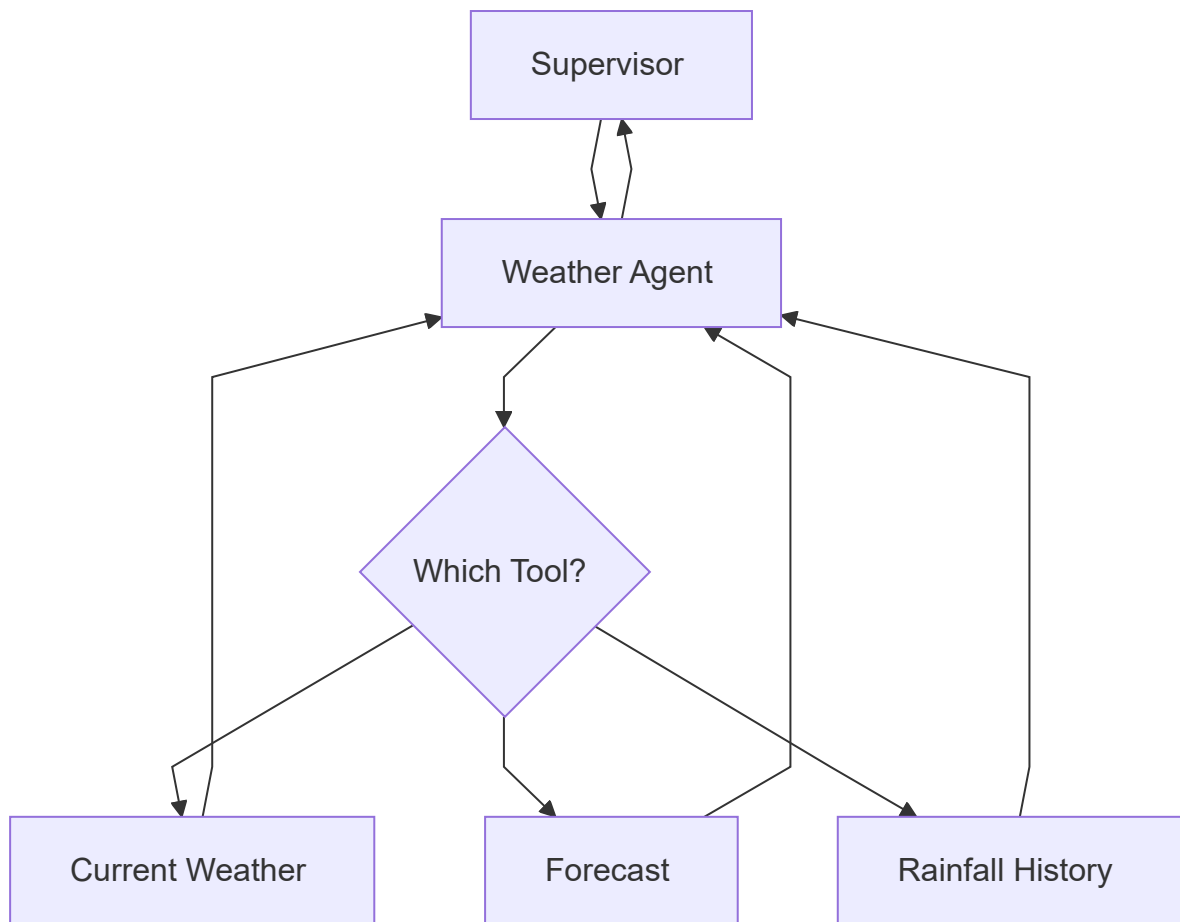
*Then the worker itself can become an agent.*

---

## 32. Agent Inside a Worker

---

*Architecture:*



*Now the Weather Agent has its own:*

LLM

Tools

Agent Loop

---

## 33. Worker as Subgraph

---



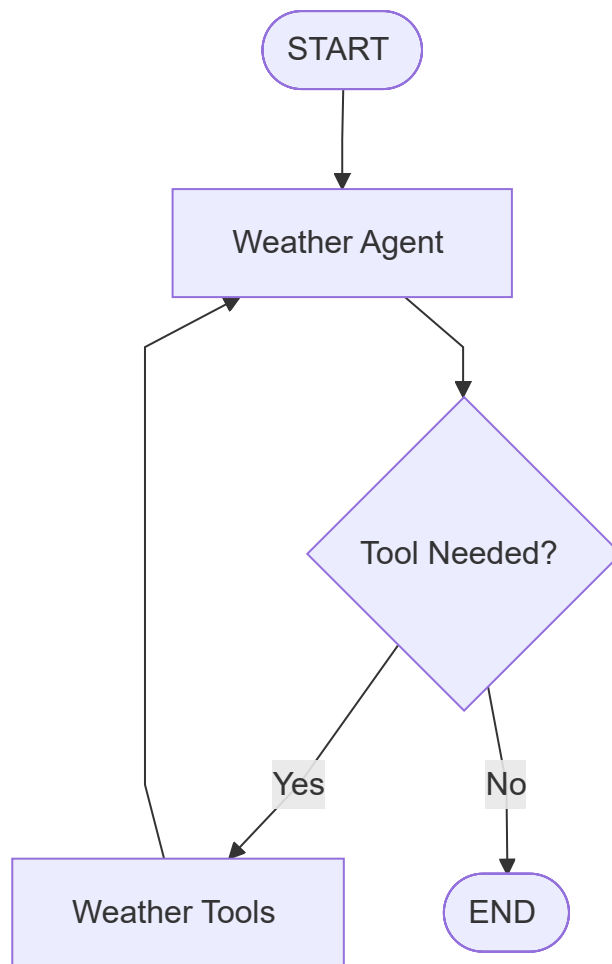
*For more complex workers:*

Main Graph

↓

weather Subgraph

*Inside:*



*Then the parent graph sees the entire workflow as:*

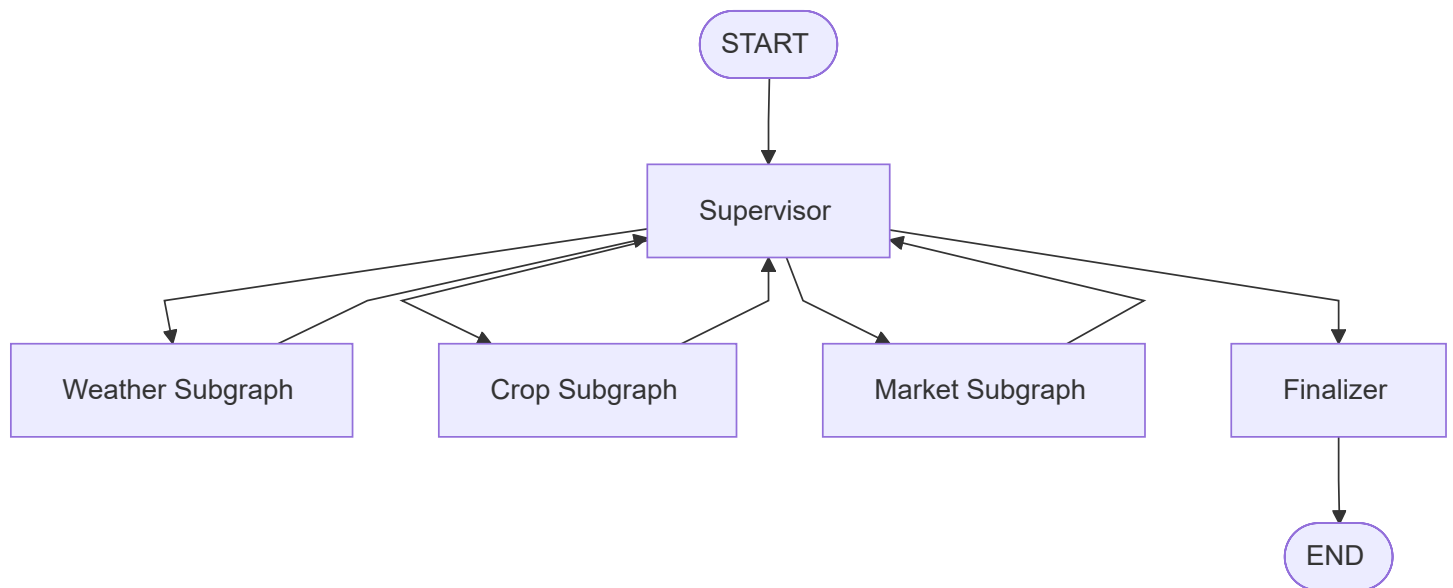
weather worker

*This keeps the main graph clean.*

---

## 34. Main Graph + Subgraphs

---



*This is a strong architecture for larger systems.*

---

## 35. Parallel Implementation

---

*Suppose every complex recommendation always needs:*

weather

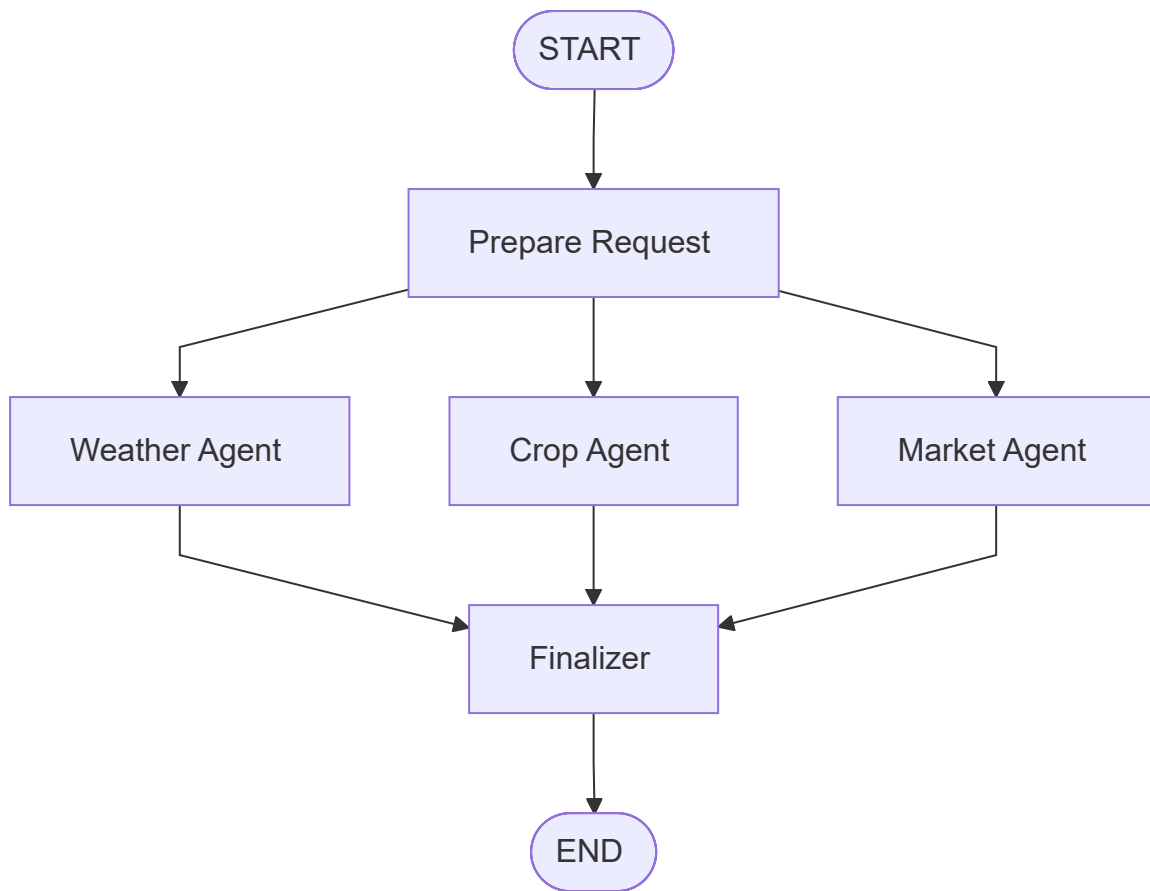
crop

market

*and these analyses are independent.*

*Then we may not need a supervisor at all.*

*Use parallel branches:*



*This is simpler and potentially faster.*

---

## 36. When Supervisor is Unnecessary

---

*If you already know:*

Every recommendation ALWAYS requires:

Weather

+

Crop

+

Market

*then asking an LLM supervisor:*



```
"what should I call next?"
```

*three times is unnecessary.*

*Use deterministic graph edges.*



*Remember:*

*If application logic already knows the workflow, encode it in the graph instead of asking an LLM to rediscover it.*

---

## 37. Deterministic vs Agentic Routing

---

*Bad:*

```
LLM:  
Should we validate the result?
```

```
LLM:  
Yes.
```

*when validation is always required.*

*Better:*

```
builder.add_edge(  
    "prediction",  
    "validation"  
)
```

*Use LLM routing only when the decision genuinely requires semantic reasoning.*

---

## 38. Sequential Implementation

---

*If dependencies exist:*

```
weather
↓
Crop
↓
Market
```

*use:*

```
builder.add_edge(
    START,
    "weather"
)

builder.add_edge(
    "weather",
    "crop"
)

builder.add_edge(
    "crop",
    "market"
)

builder.add_edge(
    "market",
    "finish"
)
```

*No supervisor required.*

## 39. Router Implementation

*Suppose user queries are domain-specific:*

```
Weather Question
OR
Crop Question
OR
Market Question
```

*Use a router:*

```
class RouteDecision(BaseModel):

    agent: Literal[
        "weather",
        "crop",
        "market"
    ]
```

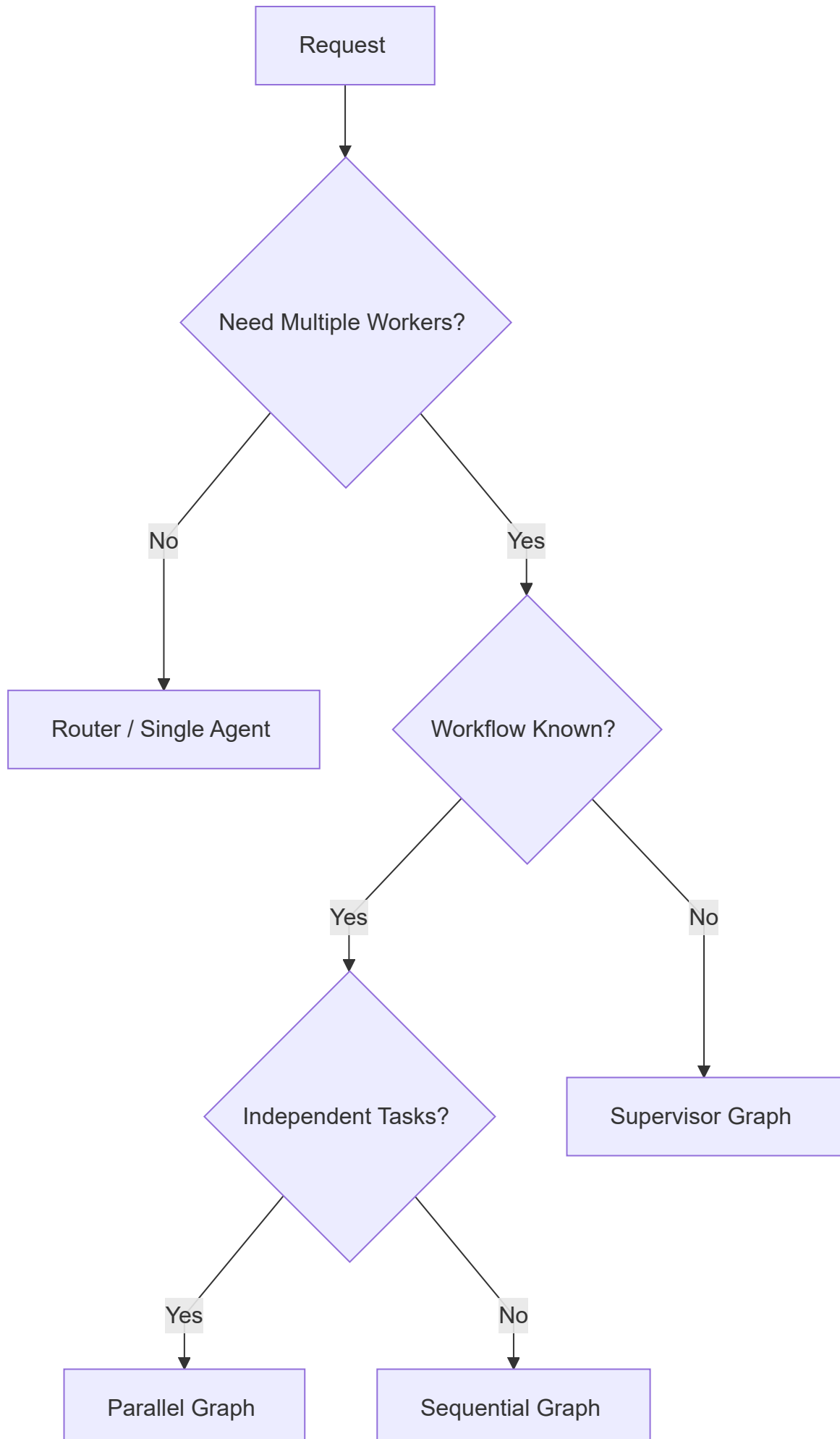
*Then:*

```
Router
↓
One specialist
↓
END
```

*Again, no supervisor loop required.*

## 40. *Choosing the Implementation*

---



*This is a very important architecture decision.*

---

## 41. Adding Persistence

---

*The main graph can use a checkpointer.*

```
from langgraph.checkpoint.memory import (
    InMemorySaver
)

checkpointer = InMemorySaver()

graph = builder.compile(
    checkpointer=checkpointer
)
```

*Then:*

```
config = {
    "configurable": {
        "thread_id":
            "conversation_123"
    }
}
```

*Now multi-agent workflow state can be persisted.*

---

## 42. Human Approval

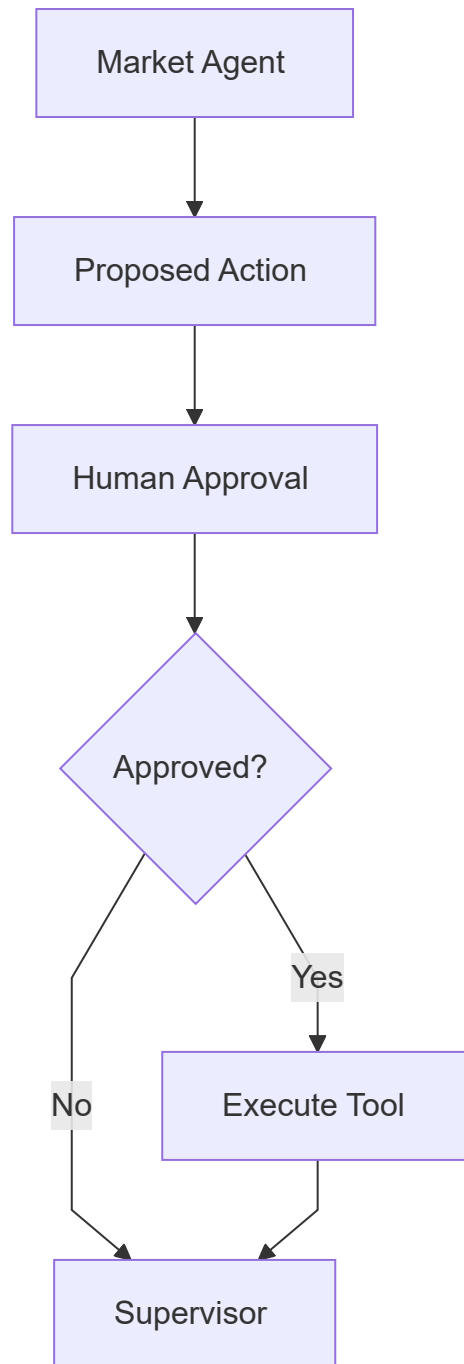
---

*Suppose the Market Agent proposes:*

Create crop sale listing

*Do not immediately execute.*

*Flow:*



*This combines:*

Multi-Agent  
+  
Human-in-the-Loop  
+  
Persistence



## 43. Error State

---

*A worker should ideally return structured failure information.*

*Example:*

```
{  
  "status":  
    "failed",  
  
  "error":  
    "weather API unavailable"  
}
```

*Instead of:*

```
Something went wrong maybe try later...
```

*The supervisor can then make a clear decision.*

---

## 44. Structured Worker Result

---

*Example:*

```
class WorkerResult(BaseModel):  
  
    status: Literal[  
        "success",  
        "failed"  
    ]  
  
    summary: str  
  
    error: str | None = None
```

*This makes worker-to-supervisor communication predictable.*

---

## 45. Agent Context Isolation

---



*Each agent should receive only the information it needs.*

*Bad:*

```
Weather Agent receives:  
  
Full conversation  
  
Market database output  
  
Crop documents  
  
Admin information  
  
All tool logs
```

*Better:*

Weather Agent receives:

User request

Location

Relevant previous results

*Benefits:*

Lower token usage

Less confusion

Better security

Cleaner specialization

---

## 46. Tool Isolation

---

*Similarly:*

Weather Agent

→ Weather Tools

*not:*

Weather Agent

→ Weather

→ Delete User

→ Update Crop

→ Payment

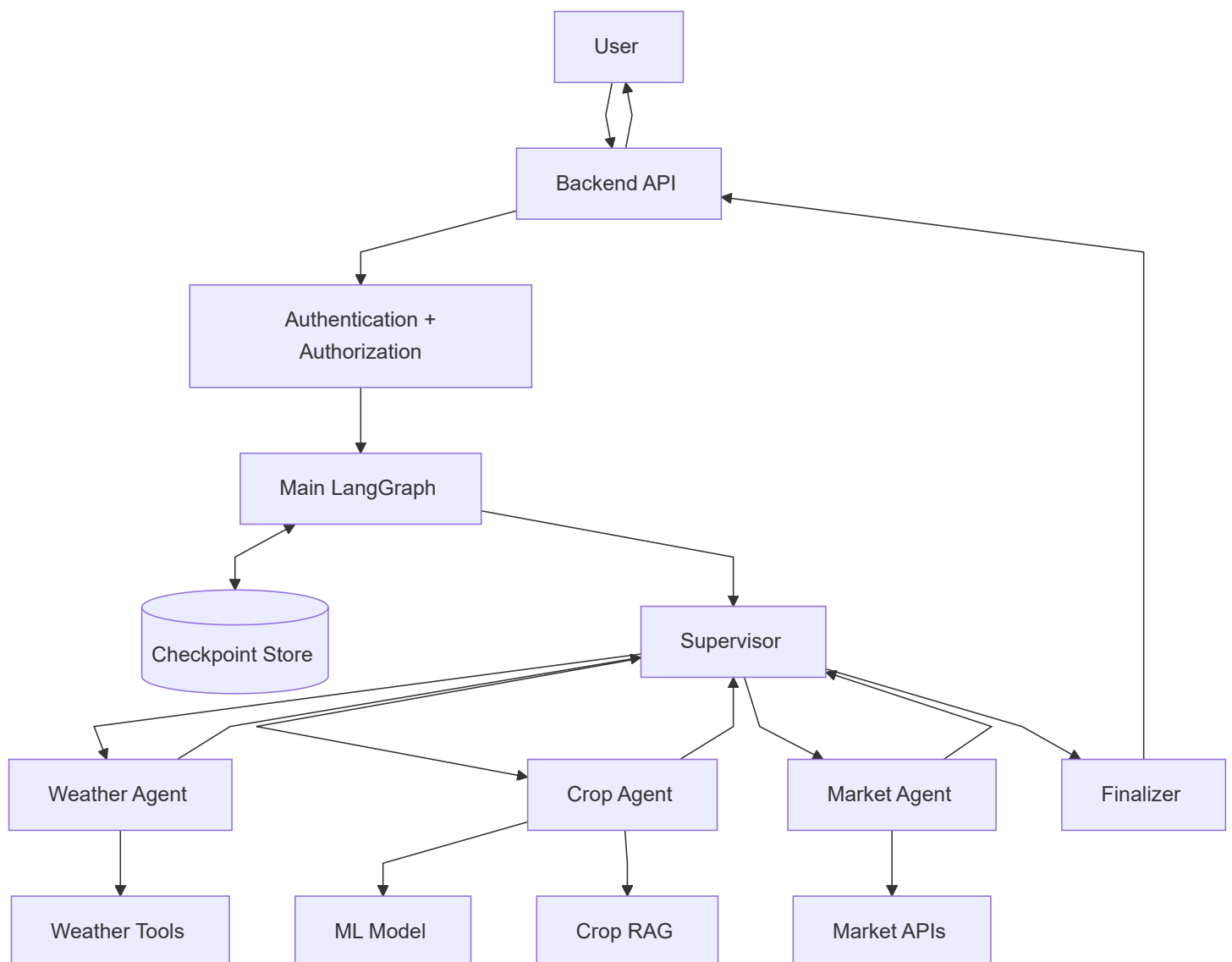
→ Admin Database

*Give agents the minimum capabilities required.*

*This follows:*

# Principle of Least Privilege

## 47. Production Architecture



# 48. Multi-Agent API Integration

---

Backend:

```
POST /agent/query
```

Request:

```
{  
  "query":  
    "Should I grow wheat?",  
  
  "location":  
    "Gorakhpur",  
  
  "crop":  
    "wheat"  
}
```

Backend:

```
Validate Request  
↓  
Authenticate User  
↓  
Create Initial Graph State  
↓  
Invoke Graph  
↓  
Return Final Answer
```

---

# 49. What Should Be Stored in Database?

---

*Do not put all application data into graph state.*

*Database:*

- Users
- Crops
- Predictions
- Conversation metadata
- Market records

*Graph state:*

- Current workflow
- Worker results
- Routing decisions
- Temporary execution information

*Checkpoint store:*

- Persisted workflow snapshots

---

# 50. Testing

---

*Test each worker separately.*

*Example:*

```
def test_weather_worker():

    state = {
        ...
    }

    result = weather_agent_node(
        state
    )

    assert (
        "weather_result"
        in result
    )
```

*Then test supervisor routing.*

```
def test_route_limit():

    state = {
        "step_count":
            MAX_STEPS,

        "next_agent":
            "weather"
    }

    assert (
        route_supervisor(state)
        == "finish"
```

## 51. Test Complete Workflow

*Important cases:*

Only Weather Needed

Weather + Crop Needed

All Agents Needed

Worker Failure

Supervisor Repetition

Maximum Step Limit

Missing User Input

Tool Failure

Finalization with Partial Results

## 52. Common Implementation Mistakes

### Mistake 1

*Creating multiple agents but giving every agent every tool.*



## Mistake 2

*Using supervisor when workflow is already deterministic.*

---

## Mistake 3

*No maximum supervisor steps.*

---

## Mistake 4

*Passing full histories to every worker.*

---

## Mistake 5

*Returning only unstructured text between workers.*

---

## Mistake 6

*Making simple API calls into separate agents.*

---

## Mistake 7

*Giving workers unnecessary permissions.*

---

## 53. Important Interview Questions

---

**Q1. How do you implement a supervisor multi-agent system in LangGraph?** ★ ★ ★ ★ ★

**Answer:**

Create a shared graph state, define a supervisor node that returns a structured next-agent decision, define specialized worker nodes, route from the supervisor using conditional edges, and return each worker to the supervisor until a finish condition is reached.

---

**Q2. How do agents communicate in LangGraph?**

**Answer:**

Agents can communicate through shared graph state, structured outputs or explicit handoffs. Shared state is useful for keeping coordination visible and predictable.

---

**Q3. Why use structured supervisor output?**



**Answer:**

It allows the LLM to make the semantic routing decision while normal application code performs deterministic graph routing using predictable values.

---

## Q4. Why do we need a supervisor step limit?

### Answer:

*The supervisor may repeatedly route between workers, so a maximum step limit prevents infinite loops, excessive cost and unnecessary latency.*

---

## Q5. Should every multi-agent system have a supervisor? ★ ★ ★ ★ ★

### Answer:

*No. Fixed workflows can use sequential edges, independent tasks can use parallel branches, and single-destination queries can use a router. A supervisor is useful when coordination genuinely needs to be dynamic.*

---

## Q6. Can each worker itself be a LangGraph?

### Answer:

*Yes. Complex workers can be implemented as subgraphs containing their own agent loops, tools and internal state.*

---

## Q7. Why isolate agent context and tools?

### Answer:

Isolation reduces token usage and confusion while improving security and specialization by giving each agent only the information and capabilities required for its responsibility.

---

## Q8. How do you handle worker failures?

### Answer:

Workers should return structured failure information so the supervisor or graph can retry, use a fallback, continue with partial results or request additional input.

---

## 54. Final Mental Model

---

Supervisor implementation:

```
START
↓
Supervisor
↓
Choose worker
↓
Worker
↓
Update Shared State
↓
Supervisor
↓
Choose Another worker
↓
...
↓
Finish
↓
```

Finalizer

↓

END

*But always ask:*

Do I really need dynamic supervision?

*If:*

only one specialist needed

*use:*

Router

*If:*

order is known

*use:*

Sequential Graph

*If:*

Tasks are independent

*use:*

Parallel Graph

*If:*

Next specialist depends on  
intermediate results

*use:*

Supervisor

*The most important implementation rule is:*

*Use deterministic LangGraph edges whenever the application already knows the workflow. Use an LLM supervisor only when deciding the next specialist genuinely requires reasoning based on the current state.*

---

## *End of Chapter 6 – Multi-Agent Systems*

---